

Princípios e exemplos de conjuntos de instruções

- An Some o número no local de armazenamento n no acumulador.
- En Se o número no acumulador for maior ou igual a zero, execute a próxima ordem que está no local de armazenamento n ; caso contrário, proceda serialmente.
- Z Pare a máquina e toque a campainha de aviso.

Wilkes e Renwick

*Seleção da Lista das 18 Instruções
de Máquina para o EDSAC (1949)*

A.1	Introdução	A-1
A.2	Classificando as arquiteturas de conjunto de instruções.....	A-2
A.3	Endereçamento de memória	A-6
A.4	Tipo e tamanho dos operandos	A-12
A.5	Operações no conjunto de instruções	A-13
A.6	Instruções para fluxo de controle.....	A-14
A.7	Codificação de um conjunto de instruções	A-18
A.8	Questões gerais: o papel dos compiladores	A-21
A.9	Juntando tudo: A arquitetura MIPS	A-29
A.10	Falácias e armadilhas	A-36
A.11	Comentários finais.....	A-40
A.12	Perspectiva histórica e referências.....	A-41
	Exercícios por Gregory D. Peterson.....	A-42

A.1 INTRODUÇÃO

Neste apêndice, nos concentraremos na arquitetura do conjunto de instruções — a parte do computador visível ao programador ou projetista de compiladores. A maior parte deste material deverá servir de revisão para os leitores; nós o incluímos aqui apenas como base. Este apêndice apresenta a grande variedade de alternativas de projeto disponíveis para o projetista do conjunto de instruções. Quatro tópicos serão enfocados: 1) vamos apresentar uma taxonomia alternativa de conjunto de instruções fixas e uma avaliação qualitativa das vantagens e desvantagens dos vários métodos; 2) vamos apresentar e analisar algumas medidas de um conjunto de instruções que são bastante independentes de um conjunto de instruções específico; 3) vamos tratar do problema das linguagens e dos compiladores e de sua relação com a arquitetura do conjunto de instruções; 4) por fim, a seção “Juntando tudo” vai mostrar como essas ideias são refletidas no conjunto de instruções MIPS, típico das arquiteturas RISC. Concluiremos com as falácias e as armadilhas do projeto do conjunto de instruções.

Para ilustrar melhor os princípios, o Apêndice K mostra quatro exemplos de arquiteturas de RISC de finalidade geral (MIPS, PowerPC, Precision Architecture, SPARC), quatro processadores de RISC embutidos (ARM, Hitachi SH, MIPS 16, Thumb) e três arquiteturas mais

antigas (80x86, IBM 360/370 e VAX). Antes de discutirmos como classificar as arquiteturas, precisamos dizer algo sobre a medida do conjunto de instruções.

Ao longo deste apêndice, examinaremos uma grande variedade de medidas arquitetônicas. Obviamente, essas medidas dependem dos programas medidos e dos compiladores que estiverem fazendo as medições. Os resultados não devem ser interpretados como absolutos, pois você poderia ver dados diferentes se fizesse a medição com um compilador ou um conjunto de programas diferente. Acreditamos que as medidas apresentadas neste apêndice são razoavelmente indicativas de uma classe de aplicações típicas. Muitas dessas medidas são apresentadas usando um pequeno conjunto de benchmarks, de forma que os dados podem ser exibidos de modo lógico e as diferenças entre os programas podem ser vistas. Diante de um computador novo, um projetista provavelmente gostaria de analisar um conjunto maior de programas antes de tomar decisões sobre a arquitetura. As medidas mostradas normalmente são *dinâmicas* — ou seja, a frequência de um evento medido é ponderada de acordo com o número de vezes que o evento acontece durante a execução do programa medido.

Antes de começar a tratar dos princípios gerais, vamos revisar as três áreas de aplicação do Capítulo 1. A *computação de desktop* enfatiza o desempenho de programas com tipos de dados de inteiro e de ponto flutuante, com pouca consideração sobre o tamanho do programa ou o consumo de energia do processador. Por exemplo, o tamanho de código nunca foi informado nas cinco gerações de benchmarks SPEC. Hoje, os *servidores* são usados principalmente como banco de dados, servidor de arquivo e aplicações de Web, assim como para algumas aplicações de compartilhamento de tempo para muitos usuários. Consequentemente, o ponto flutuante é muito menos importante para o desempenho do que os inteiros e strings de caracteres, embora quase todo processador ainda inclua instruções de ponto flutuante. As *aplicações embarcadas* avaliam custo e energia; assim, o tamanho de código é importante porque menos memória é sinônimo de memória mais barata e que consome menos energia; além disso, algumas classes de instrução (como ponto flutuante) podem ser opcionais para reduzir custos de chip.

Portanto, os conjuntos de instruções para as três aplicações são muito semelhantes. Na verdade, a arquitetura MIPS que enseja este apêndice foi usada com sucesso em aplicações de desktop, servidores e embarcadas.

Uma arquitetura bem-sucedida muito diferente da RISC é a 80x86 (Apêndice K). Surpreendentemente, seu sucesso não desmente necessariamente as vantagens de um conjunto de instruções RISC. A importância comercial da compatibilidade binária com o software de PC, combinada com a abundância de transistores fornecida pela lei de Moore, levou a Intel a usar um conjunto de instruções RISC internamente enquanto aceitava um conjunto de instruções 80x86 externamente. Os microprocessadores 80x86 recentes, como o Pentium 4, usam hardware para traduzir instruções 80x86 para instruções tipo RISC e então executar as operações traduzidas dentro do chip. Eles mantêm a ilusão da arquitetura 80x86 para o programador enquanto permitem ao projetista de computador implementar um processador do tipo RISC para desempenho.

Agora que a base está preparada, começamos explorando o modo como as arquiteturas de conjunto de instruções podem ser classificadas.

A.2 CLASSIFICANDO AS ARQUITETURAS DE CONJUNTO DE INSTRUÇÕES

O tipo de armazenamento interno em um processador é a diferenciação mais básica; portanto, nesta seção, enfocaremos as alternativas para essa parte da arquitetura. As principais escolhas são: uma pilha, um acumulador ou um conjunto de registradores. Os operandos

podem ser nomeados explícita ou implicitamente: os operandos em uma *arquitetura de pilha* estão implicitamente no topo da pilha e, em uma *arquitetura de acumulador*, um operando é implicitamente o acumulador. As *arquiteturas de registradores de propósito geral* possuem apenas operandos explícitos — registradores ou locais de memória. A [Figura A.1](#) mostra um diagrama de bloco dessas arquiteturas, e a [Figura A.2](#) mostra como a

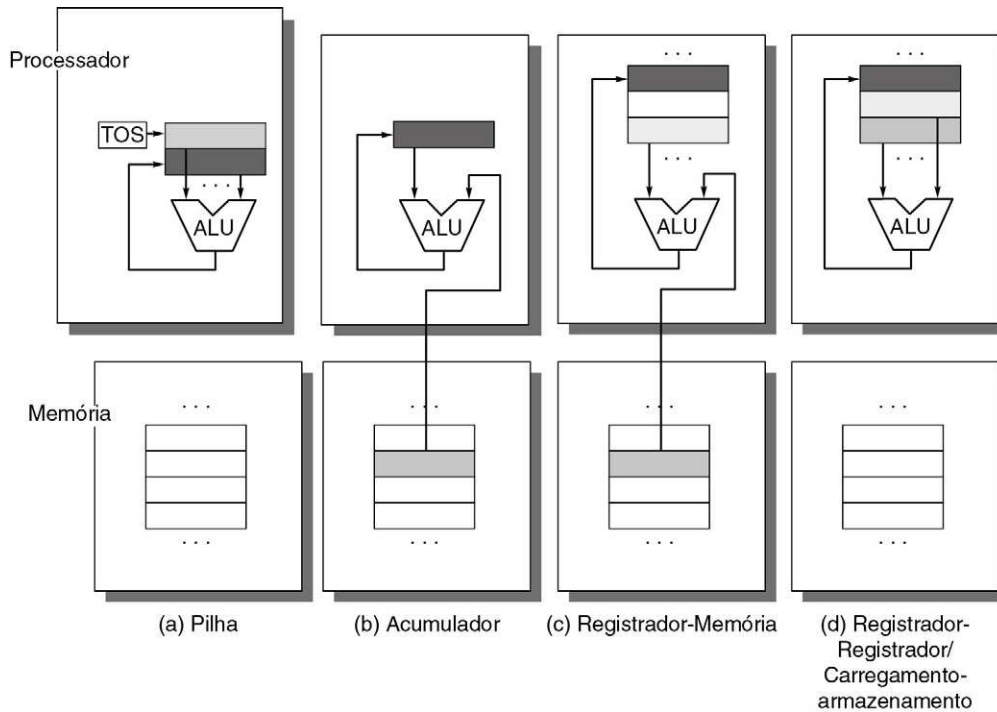


FIGURA A.1 Locais de operandos para as quatro classes de arquitetura de conjunto de instruções.

As setas indicam se o operando é uma entrada ou o resultado da operação da ULA ou se é ao mesmo tempo uma entrada e o resultado. Os sombreados claros indicam entradas, e o sombreado mais escuro indica o resultado. Em (a), um registrador Top Of Stack (TOS) aponta para o operando de entrada superior, que é combinado com o operando abaixo. O primeiro operando é removido da pilha, o resultado assume o lugar do segundo operando, e o TOS é atualizado para apontar para o resultado. Todos os operandos estão implícitos. Em (b), o acumulador é tanto um operando de entrada implícito quanto um resultado. Em (c), um operando de entrada é um registrador, um está em memória e o resultado vai para um registrador. Todos os operandos são registradores em (d) e, como a arquitetura de pilha, só pode ser transferido para a memória através de instruções separadas: push ou pop para (a) e carregamento ou armazenamento para (d).

Pilha	Acumulador	Registrador (registrador-memória)	Registrador (carregamento-armazenamento)
Push A	Load A	Load R1,A	Load R1,A
Push B	Add B	Add R3,R1,B	Load R2,B
Add	Store C	Store R3,C	Add R3,R1,R2
Pop C			Store R3,C

FIGURA A.2 A sequência de código para $C = A + B$ para quatro classes de conjuntos de instruções.

Observe que a instrução Add possui operandos implícitos para arquiteturas de pilha e de acumulador, e operandos explícitos para arquiteturas de registrador. Considera-se que A, B e C pertençam à memória e que os valores de A e B não podem ser destruídos. A [Figura A.1](#) mostra a operação Add para cada classe da arquitetura.

sequência de código $C = A + B$ normalmente se pareceria nessas três classes de conjuntos de instruções. Os operandos explícitos podem ser acessados diretamente da memória ou precisar ser primeiramente carregados no armazenamento temporário, dependendo da classe da arquitetura e da escolha da instrução específica.

Como mostram as figuras, na realidade, há duas classes de registradores: uma classe que pode acessar a memória como parte de qualquer instrução, chamada arquitetura *registrador-memória*, e a outra que só pode acessar a memória com instruções de carregamento (load) e armazenamento (store), chamada de arquitetura carregamento-armazenamento (load-store) ou registrador-registrador. Uma terceira classe, não encontrada em computadores modernos, mantém todos os operandos na memória e é chamada arquitetura *memória-memória*. Algumas arquiteturas de conjunto de instruções possuem mais registros do que um único acumulador, mas impõem restrições sobre os usos desses registradores especiais. Às vezes tal arquitetura é chamada *acumulador estendido* ou computador *registrador de propósito específico*.

Embora a maioria dos primeiros computadores usasse arquiteturas de pilha ou de acumulador, quase todas as novas arquiteturas projetadas após 1980 usam uma arquitetura de registrador de carregamento-armazenamento. São duas as principais razões para o surgimento dos computadores com registradores de propósito geral (GPR): 1) os registradores, assim como outras formas de armazenamento internas ao processador, são mais rápidos do que a memória; 2) para um compilador, usar registradores é mais eficiente do que empregar outras formas de armazenamento interno. Por exemplo, em um computador de registrador, a expressão $(A * B) - (B * C) - (A * D)$ pode ser avaliada efetuando-se as multiplicações em qualquer ordem, o que pode ser mais eficiente, devido ao local dos operandos ou às preocupações com o pipelining (Cap. 2). Entretanto, em um computador de pilha, o hardware precisa avaliar a expressão apenas em uma ordem, já que os operandos são ocultos na pilha e pode ser necessário carregar um operando diversas vezes.

Mais importante ainda, podem ser usados os registradores para armazenar as variáveis. Quando as variáveis são alocadas em registradores, o tráfego de memória reduz, o programa acelera (já que os registradores são mais rápidos que a memória) e a densidade de código melhora (já que um registrador pode ser nomeado com menos bits do que um local de memória).

Como explicado na [Seção A.8](#), os projetistas de compilador prefeririam que todos os registradores fossem equivalentes e não reservados. Os computadores mais antigos comprometem esse desejo dedicando registradores a usos especiais, diminuindo efetivamente o número de registradores de propósito geral. Se o número de registradores de propósito geral for muito pequeno, tentar alocar variáveis para registradores não será proveitoso. Em vez disso, o compilador reservará todos os registradores não comprometidos para uso na avaliação de expressões.

Quantos registradores são suficientes? A resposta, é claro, depende da eficiência do compilador. A maioria dos compiladores reserva alguns registradores para avaliação de expressões, usa alguns para passagem de parâmetros e permite que o restante seja alocado para armazenar variáveis. A tecnologia de compiladores modernos e sua capacidade de usar eficientemente um número maior de registradores levaram a um aumento no número de registradores nas arquiteturas mais novas.

Duas grandes características de conjuntos de instruções dividem as arquiteturas GPR. Essas duas características se relacionam à natureza dos operandos para uma aritmética ou lógica típica (instrução da ULA). A primeira é sobre se uma instrução da ULA tem dois ou três operandos. No formato de três operandos, a instrução contém um operando de resultado e dois operandos de origem. No formato de dois operandos, um dos operandos é, ao mesmo tempo, uma origem e um resultado para a operação. A segunda distinção entre as arquiteturas GPR se refere a quantos dos operandos podem ser endereços de memória

nas instruções da ULA. O número de operandos de memória aceito por uma instrução da ULA típica pode variar de zero a três. A [Figura A.3](#) mostra combinações desses dois atributos com exemplos de computadores. Embora haja sete combinações possíveis, três servem para classificar quase todos os computadores existentes. Como já mencionamos, essas três são carregamento-armazenamento (também chamada registrador-registrador), registrador-memória e memória-memória.

A [Figura A.4](#) mostra as vantagens e desvantagens de cada uma dessas alternativas. Sem dúvida, essas vantagens e desvantagens não são absolutas: elas são qualitativas e seu verdadeiro impacto depende do compilador e da estratégia de implementação. Um computador GPR com operações memória-memória poderia ser facilmente ignorado pelo compilador e usado como um computador de carregamento-armazenamento. Um dos impactos arquitetônicos mais penetrantes é sobre a codificação de instrução e o número de instruções necessárias para executar uma tarefa. Examinamos o impacto dessas alternativas arquitetônicas sobre os métodos de implementação no Apêndice C e no Capítulo 3.

Número de endereços de memória	Número máximo de operandos permitidos	Tipo de arquitetura	Exemplos
0	3	Carregamento-armazenamento/ registrador-registrador	Alpha, ARM, MIPS, PowerPC, SPARC, SuperH, TM32
1	2	Registrador-memória	IBM 360/370, Intel 80x86, Motorola 68000, TI TMS320C54x
2	2	Memória-memória	VAX (também possui formatos de três operandos)
3	3	Memória-memória	VAX (também possui formatos de dois operandos)

FIGURA A.3 Combinações típicas de operandos de memória e totais de operandos por instruções típicas da ULA com exemplos de computadores.

Os computadores sem referência à memória por instrução da ULA são chamados de computadores de carregamento-armazenamento ou registrador-registrador. As instruções com vários operandos de memória por instruções típicas da ULA são chamadas de registrador-memória ou memória-memória, conforme tenham um ou mais de um operando de memória.

Tipo	Vantagens	Desvantagens
Registrador-registrador (0, 3)	Codificação de instruções simples e de tamanho fixo. As instruções são executadas com um número de clocks semelhantes (Apêndice C).	Contagem de instrução mais alta do que as arquiteturas com referências à memória nas instruções. Mais instruções e densidade de instrução mais baixa produzem programas maiores.
Registrador-memória (1, 2)	Os dados podem ser acessados sem executar uma instrução de carregamento (load) primeiro. O formato de instrução costuma ser fácil de codificar e apresenta boa densidade.	Os operandos não são equivalentes, já que um operando de origem em uma operação binária é destruído. Codificar um número de registrador e um endereço de memória em cada instrução pode restringir o número de registradores. O número de clocks por instrução varia conforme o local do operando.
Memória-memória (2, 2) ou (3, 3)	Mais compacto. Não desperdiça registradores para temporários.	Grande variação no tamanho de instrução, especialmente para instruções de três operandos. Além disso, grande variação no trabalho por instrução. Os acessos à memória criam gargalos de memória. (Não usado atualmente.)

FIGURA A.4 Vantagens e desvantagens dos três tipos mais comuns de computadores registradores de finalidade geral.

A notação (m, n) significa m operandos de memória e n operandos totais. Em geral, os computadores com menos alternativas simplificam a tarefa do compilador, já que existem menos decisões para o compilador tomar ([Seção A.8](#)). Os computadores com grande variedade de formatos de instrução flexíveis reduzem o número de bits necessários para codificar o programa. O número de registradores também afeta o tamanho da instrução, já que $\log_2$ (número de registradores) é necessário para cada especificador de registrador em uma instrução. Portanto, dobrar o número de registrador consome 3 bits extras para uma arquitetura registrador-registrador ou aproximadamente 10% de uma instrução de 32 bits.

Resumo: classificando as arquiteturas de conjunto de instruções

Aqui e no final das Seções A.3 a A.8, resumimos as características que esperaríamos encontrar em uma nova arquitetura de conjunto de instruções, construindo os fundamentos para a arquitetura MIPS introduzida na Seção A.9. A partir desta seção, devemos claramente esperar o uso de registradores de propósito geral. A Figura A.4, combinada com o Apêndice C sobre pipelining, leva à expectativa de uma versão carregamento-armazenamento (load-store) de uma arquitetura de registrador de propósito geral.

Com a classe da arquitetura discutida, o próximo tópico é o endereçamento dos operandos.

A.3 ENDEREÇAMENTO DE MEMÓRIA

Quer a arquitetura seja carregamento-armazenamento, quer permita que qualquer operando seja uma referência à memória, ela precisa definir como os endereços de memória são interpretados e como são especificados. As medidas apresentadas aqui são, em grande parte — mas não completamente — independentes do computador. Em alguns casos, as medidas são significativamente afetadas pela tecnologia do compilador. Essas medidas foram feitas usando um compilador de otimização, já que a tecnologia de compiladores desempenha um papel vital.

Interpretando endereços de memória

Como um endereço de memória é interpretado? Ou seja, que objeto é acessado como uma função do endereço e do comprimento? Todos os conjuntos de instruções discutidos neste livro são endereçados por byte e fornecem acesso para bytes (8 bits), meias palavras (16 bits) e palavras (32 bits). A maioria dos computadores também fornece acesso para palavras duplas (64 bits).

Existem duas convenções diferentes para ordenar os bytes dentro de um objeto maior. A ordem de bytes *Little Endian* coloca o byte cujo endereço é “x... x000” na posição menos significativa da palavra dupla (a extremidade de menor ordem). Os bytes são numerados:

7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---

A ordem de byte *Big Endian* coloca o byte cujo endereço é “x... x000” na posição mais significativa da palavra dupla (extremidade de maior ordem). Os bytes são numerados:

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

Ao operar dentro de um computador, a ordem de byte costuma ser imperceptível — apenas programas que acessam os mesmos locais, como palavras e bytes, por exemplo, podem notar a diferença. Entretanto, a ordem de bytes é um problema ao trocar dados entre computadores com ordenações diferentes. A ordenação *Little Endian* também não corresponde à ordenação normal de palavras quando strings são comparadas. As strings aparecem ao “OIRÁRTNOC” (ao contrário) nos registros.

Um segundo aspecto da memória é que, em muitos computadores, os acessos a objetos maiores que um byte devem estar *alinhados*. Um acesso a um objeto de tamanho s bytes no endereço de byte A é alinhado se $A \bmod s = 0$. A Figura A.5 mostra os endereços em que um acesso está alinhado ou desalinhado.

Por que alguém projetaria um computador com restrições de alinhamento? O desalinhamento causa complicações de hardware, já que a memória normalmente é alinhada

em um múltiplo de um limite de uma palavra ou palavra dupla. Um acesso à memória desalinhado, portanto, pode envolver múltiplas referências de memória alinhadas. Assim, até mesmo em computadores que permitem acesso desalinhado, os programas com acessos alinhados são executados mais rapidamente.

Mesmo se os dados estiverem alinhados, o suporte a byte, meia palavra e palavra exige uma rede de alinhamento para alinhar bytes, meias palavras e palavras em registradores de 64 bits. Por exemplo, na [Figura A.5](#), suponha a leitura de um byte de um endereço com seus 3 bits de baixa ordem tendo o valor 4. Precisaremos deslocar 3 bytes para a direita para alinhar o byte com o lugar correto em um registrador de 64 bits. Dependendo da instrução, o computador também pode precisar estender o sinal. Os armazenamentos são fáceis: apenas os bytes endereçados na memória podem ser alterados. Em alguns computadores, uma operação de byte, meia palavra e palavra não afeta a parte superior de um registrador. Embora todos os computadores discutidos neste livro permitam acessos de byte, meia palavra e palavra à memória, apenas o IBM 360/370, o Intel 80x86 e o VAX aceitam operações da ULA em operandos de registradores menores que a largura total.

Agora que discutimos as interpretações alternativas dos endereços de memória, podemos discutir as maneiras como os endereços são especificados por instruções, chamadas *modos de endereçamento*.

Modos de endereçamento

Dado um endereço, sabemos que bytes acessar na memória. Nesta subseção, veremos os modos de endereçamento — como as arquiteturas especificam o endereço de um objeto

Valor dos 3 bits de ordem inferior do endereço de byte								
Largura do objeto	0	1	2	3	4	5	6	7
1 byte (byte)	Alinhado	Alinhado	Alinhado	Alinhado	Alinhado	Alinhado	Alinhado	Alinhado
2 bytes (meia palavra)	Alinhado		Alinhado		Alinhado		Alinhado	
2 bytes (meia palavra)		Desalinhado		Desalinhado		Desalinhado		Desalinhado
4 bytes (palavra)	Alinhado				Alinhado			
4 bytes (palavra)		Desalinhado				Desalinhado		
4 bytes (palavra)		Desalinhado					Desalinhado	
4 bytes (palavra)		Desalinhado						Desalinhado
8 bytes (palavra dupla)	Alinhado							
8 bytes (palavra dupla)		Desalinhado						
8 bytes (palavra dupla)		Desalinhado						
8 bytes (palavra dupla)		Desalinhado						
8 bytes (palavra dupla)		Desalinhado						
8 bytes (palavra dupla)		Desalinhado						
8 bytes (palavra dupla)		Desalinhado						
8 bytes (palavra dupla)		Desalinhado						
8 bytes (palavra dupla)		Desalinhado						

FIGURA A.5 Endereços alinhados e desalinhados de objetos com tamanho de byte, meia palavra, palavra e palavra dupla para computadores endereçados por byte.

Para cada exemplo desalinhado, alguns objetos exigem dois acessos à memória para serem completados. Cada objeto alinhado sempre pode ser completado em um acesso à memória, desde que a memória seja tão grande quanto o objeto. A figura mostra a memória organizada como 8 bytes de extensão. Os deslocamentos de byte que rotulam as colunas especificam os 3 bits de ordem inferior do endereço.

que elas acessarão. Os modos de endereçamento especificam constantes e registradores, além de posições na memória. Quando um local da memória é usado, o endereço de memória real especificado pelo modo de endereçamento é chamado *endereço efetivo*.

A Figura A.6 mostra todos os modos de endereçamento de dados que foram usados em computadores modernos. Imediatos ou literais normalmente são considerados modos de endereçamento de memória (ainda que o valor que acessam esteja no fluxo de instrução), embora os registradores frequentemente sejam separados, já que eles normalmente não possuem endereços de memória. Temos mantido separados os modos de endereçamento que dependem do contador de programa, chamado *endereçamento relativo ao PC*. O endereçamento relativo ao PC é usado principalmente para especificar endereços de código em instruções de transferência de controle, discutidas na Seção A.6.

A Figura A.6 mostra os nomes mais comuns para os modos de endereçamento, embora os nomes difiram entre as arquiteturas. Nessa figura e em todo o livro, usamos uma extensão para a linguagem de programação C como uma notação de descrição de hardware.

Modo de endereçamento	Instrução de exemplo	Significado	Quando é usado
Registrador	Add R4, R3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Regs}[R3]$	Quando um valor está em um registrador.
Imediato	Add R4, #3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + 3$	Para constantes.
Deslocamento	Add R4, 100(R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[100 + \text{Regs}[R1]]$	Ao acessar variáveis locais (+ simula modos de endereçamento indireto de registrador e direto).
Indireto de registrador	Add R4, (R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[\text{Regs}[R1]]$	Ao acessar usando um ponteiro ou um endereço calculado.
Indexado	Add R3, (R1+R2)	$\text{Regs}[R3] \leftarrow \text{Regs}[R3] + \text{Mem}[\text{Regs}[R1] + \text{Regs}[R2]]$	Algumas vezes útil no endereçamento de array: R1 = base do array; R2 = valor de índice.
Direto ou absoluto	Add R1, (1001)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[1001]$	Algumas vezes útil para acessar dados estáticos; a constante do endereço pode precisar ser grande.
Indireto de memória	Add R1, @(R3)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Mem}[\text{Regs}[R3]]]$	Se R3 é o endereço de um ponteiro p , então o modo produz $*p$.
Autoincremento	Add R1, (R2)+	$\text{Regs}[R2] \leftarrow \text{Regs}[R2] + d$	Útil para percorrer os arrays passo a passo dentro de um loop. R2 aponta para o início do array; cada referência incrementa R2 pelo tamanho de um elemento, d .
Autodecremento	Add R1, -(R2)	$\text{Regs}[R2] \leftarrow \text{Regs}[R2] - d$ $\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]]$	Mesmo uso do autoincremento. Autodecremento/autoincremento também podem agir como push/pop para implementar uma pilha.
Escalonado	Add R1, 100(R2)[R3]	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[100 + \text{Regs}[R2] + \text{Regs}[R3] * d]$	Usado para indexar arrays. Pode ser aplicado para qualquer modo de endereçamento indexado em alguns computadores.

FIGURA A.6 Seleção dos modos de endereçamento com exemplos, significado e uso.

Nos modos de endereçamento escalonado e de autoincremento/autodecremento, a variável d indica o tamanho do item de dados sendo acessado (ou seja, se a instrução está acessando 1, 2, 4 ou 8 bytes). Esses modos de endereçamento são úteis apenas quando os elementos que estão sendo acessados são adjacentes na memória. Os computadores RISC usam endereçamento de deslocamento para simular indireto de registrador com 0 como o endereço e para simular endereçamento direto usando 0 no registrador de base. Em nossas medições, usamos o primeiro nome mostrado para cada modo. As extensões para C usadas como descrições de hardware são definidas na página A-32.

Nessa figura, apenas um recurso não C é usado: a seta para a esquerda ($\leftarrow$) é usada para atribuição. Também usamos o array `Mem` como o nome para a memória principal e o array `Regs` para registradores. Portanto, `Mem[Regs[R1]]` se refere ao conteúdo do local de memória cujo endereço é dado pelo conteúdo do registrador 1 (`R1`). Mais adiante, introduziremos extensões para acessar e transferir dados menores que uma palavra.

Os modos de endereçamento possuem a capacidade de reduzir significativamente as contagens de instrução; eles também aumentam a complexidade de construir um computador e podem elevar o CPI (ciclos de clock por instrução) médio dos computadores que implementam esses modos. Assim, o uso de vários modos de endereçamento é muito importante para ajudar o projetista a escolher o que incluir.

A [Figura A.7](#) mostra os resultados de medir padrões de uso de modos de endereçamento em três programas na arquitetura VAX. Usamos a velha arquitetura VAX para algumas medições neste apêndice porque ela tem o conjunto mais rico de modos de endereçamento e o mínimo de restrições sobre o endereçamento de memória. Por exemplo, a [Figura A.6](#) na página A-8 mostra todos os modos que o VAX aceita. A maioria das medições neste apêndice, contudo, usará as arquiteturas registrador-registrador mais recentes para mostrar como os programas usam conjuntos de instruções de computadores atuais.

Como mostra a [Figura A.7](#), o endereçamento imediato e o de deslocamento dominam o uso dos modos de endereçamento. Vejamos algumas propriedades desses dois modos amplamente utilizados.

Modo de endereçamento de deslocamento

A principal questão que surge para um modo de endereçamento no estilo de deslocamento é a da faixa de deslocamentos usada. Com base no uso de vários tamanhos de deslocamen-

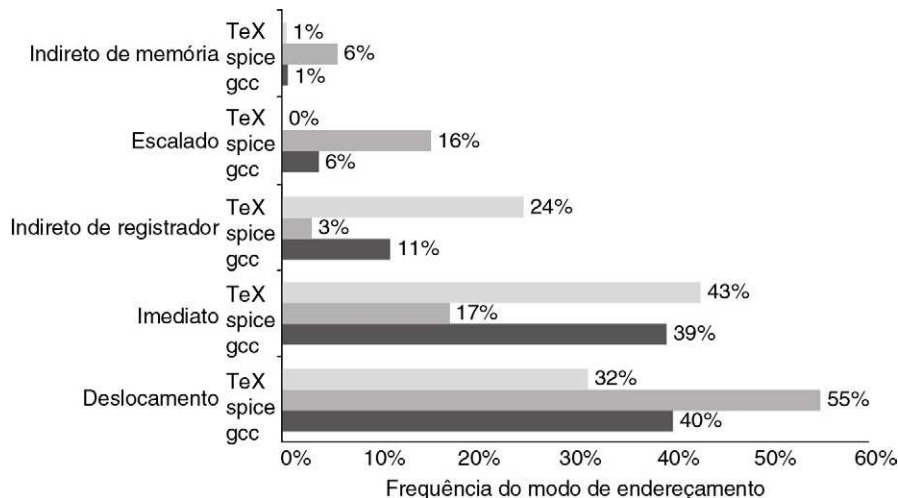


FIGURA A.7 Resumo do uso dos modos de endereçamento de memória (incluindo imediatos).

Esses principais modos de endereçamento são responsáveis por apenas 0-3% dos acessos à memória.

Os modos de registrador, que não são contados, respondem por metade das referências de operando, enquanto os modos de endereçamento de memória (incluindo imediato) respondem pela outra metade. Obviamente, o compilador afeta os modos de endereçamento que são usados; veja a [Seção A.8](#). O modo indireto de memória no VAX pode usar deslocamento, autoincremento ou autodecremento para formar o endereço de memória inicial; nesses programas, quase todas as referências indiretas à memória usam o modo de deslocamento como base. O modo de deslocamento inclui todos os tamanhos de deslocamento (8, 16 e 32 bits). Os modos de endereçamento relativos ao PC, usados quase exclusivamente para desvios, não estão incluídos. Apenas os modos de endereçamento com frequência média de mais de 1% são mostrados.

to, uma decisão de que tamanhos serão aceitos pode ser tomada. É importante escolher os tamanhos de campo de deslocamento porque eles afetam diretamente o tamanho da instrução. A [Figura A.8](#) mostra as medições feitas no acesso a dados em uma arquitetura carregamento-armazenamento usando nossos programas de benchmark. Examinamos os offsets de desvio na [Seção A.6](#) — os padrões de acesso a dados e desvios são diferentes; há pouca vantagem em combiná-los, embora, na prática, os tamanhos dos imediatos sejam considerados iguais por simplicidade.

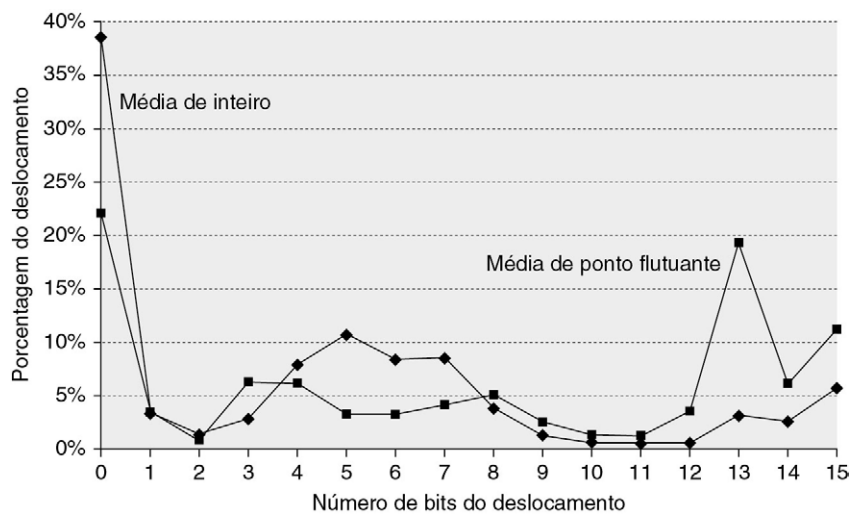


FIGURA A.8 Os valores de deslocamento são largamente distribuídos.

Existe grande número de valores pequenos e elevado número de valores grandes. A ampla distribuição dos valores de deslocamento é devida às múltiplas áreas de armazenamento para variáveis e diferentes deslocamentos para acessá-las ([Seção A.8](#)), bem como ao esquema de endereçamento geral que o compilador usa. O eixo x é $\log_2$ do deslocamento, ou seja, o tamanho de um campo necessário para representar a magnitude do deslocamento. O zero no eixo x mostra a porcentagem dos deslocamentos do valor 0. O gráfico não inclui o bit de sinal, que é enormemente afetado pelo leiaute do armazenamento. A maioria dos deslocamentos é positiva, mas uma minoria dos deslocamentos maiores (mais de 14 bits) é negativa. Como esses dados foram coletados em um computador com deslocamentos de 16 bits, eles não podem nos dizer sobre deslocamentos maiores. Esses dados foram tirados em uma arquitetura Alpha com total otimização ([Seção A.8](#)) sobre o SPEC CPU2000, mostrando a média dos programas de inteiro (CINT2000) e a média dos programas de ponto flutuante (CFP2000).

Modo de endereçamento imediato ou literal

Os imediatos podem ser usados nas operações aritméticas, em comparações (principalmente para desvios) e em movimentos onde uma constante é necessária em um registrador. O último caso ocorre para constantes escritas no código, que tendem a ser pequenas, e para constantes de endereço, que costumam ser grandes. Para uso dos imediatos, é importante saber se eles precisam ser aceitos para todas as operações ou apenas para um subconjunto. A [Figura A.9](#) mostra a frequência dos imediatos para as classes gerais das operações de inteiro e ponto flutuante em um conjunto de instruções.

Outra importante medição de conjunto de instruções é a faixa de valores para imediatos. Assim como os valores de deslocamento, o tamanho dos valores de imediato afeta o tamanho da instrução. Como mostra a [Figura A.10](#), valores de imediato pequenos são extensamente usados. Imediatos grandes, no entanto, algumas vezes são usados, mais provavelmente em cálculos de endereçamento.

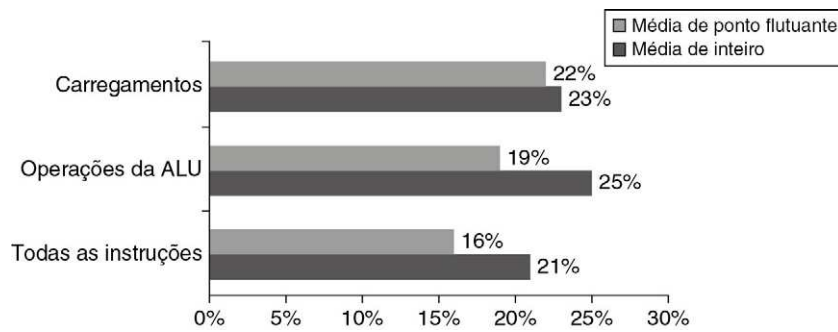


FIGURA A.9 Aproximadamente 1/4 das transferências de dados e operações da ULA possui um operando imediato.

As barras inferiores mostram que os programas de inteiro usam immediatos em cerca de 1/5 das instruções, enquanto os programas de ponto flutuante usam immediatos em aproximadamente 1/6 das instruções. Para carregamentos, a instrução de carregamento imediato carrega 16 bits em uma das metades de um registrador de 32 bits. Os carregamentos immediatos não são carregamentos em um sentido estrito porque não acessam a memória. Ocasionalmente, um par de carregamentos immediatos é usado para carregar uma constante de 32 bits, mas isso é raro. (Para operações da ULA, deslocamentos por um valor constante são incluídos como operações com operandos immediatos.) Os programas e o computador usados para coletar essas estatísticas são iguais aos da [Figura A.8](#).

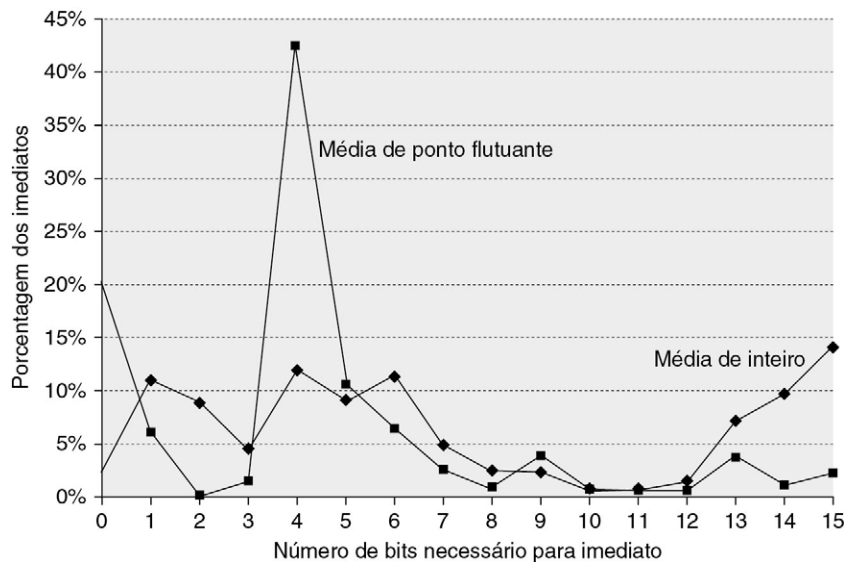


FIGURA A.10 Distribuição de valores immediatos.

O eixo x mostra o número de bits necessário para representar a magnitude de um valor imediato — 0 significa que o valor do campo imediato era 0. A maioria dos valores immediatos é positiva. Cerca de 20% foram negativos para CINT2000 e aproximadamente 30% foram negativos para CFP2000. Essas medições foram realizadas no Alpha, onde o imediato máximo é 16 bits, para os mesmos programas da [Figura A.8](#). Uma medição semelhante no VAX, que aceitava immediatos de 32 bits, mostrou que cerca de 20-25% dos immediatos eram mais longos que 16 bits. Portanto, 16 bits capturariam aproximadamente 80% e 8 bits cerca de 50%.

Resumo: endereçamento de memória

Primeiro, devido à sua popularidade, esperaríamos que uma nova arquitetura aceitasse pelo menos os seguintes modos de endereçamento: deslocamento, imediato e indireto de registrador. A [Figura A.7](#) mostra que eles representam 75-99% dos modos de endereçamento usados em nossas medições. Segundo, esperaríamos que o tamanho do endereço para o modo de deslocamento fosse pelo menos 12-16 bits, já que a legenda na [Figura A.8](#) sugere que esses tamanhos cobririam 75-99% dos deslocamentos. Terceiro, esperaríamos que o tamanho do campo imediato fosse de pelo menos 8-16 bits, pois cobririam 50-80% dos immediatos.

Tendo coberto as classes de conjunto de instruções e decidido pelas arquiteturas registrador-registrador, além das recomendações anteriores sobre os modos de endereçamento de dados, examinaremos a seguir os tamanhos e as medições dos dados.

A.4 TIPO E TAMANHO DOS OPERANDOS

Como é designado o tipo de um operando? Normalmente, codificar no opcode designa o tipo de operando — esse é o método mais utilizado. Alternativamente, os dados podem ser anotados com tags que são interpretadas pelo hardware. Essas tags especificam o tipo do operando, e a operação é escolhida de acordo. Os computadores com dados identificados por tags, no entanto, só podem ser encontrados em museus.

Vamos começar com as arquiteturas de desktop e de servidores. Em geral, o tipo de operando — inteiro, ponto flutuante de precisão simples, caractere etc. — efetivamente fornece o tamanho. Tipos de operando comuns incluem caractere (8 bits), meia palavra (16 bits), palavra (32 bits), ponto flutuante de precisão simples (também uma palavra) e ponto flutuante de precisão dupla (duas palavras). Inteiros são quase universalmente representados como números binários em complemento de dois. Os caracteres normalmente estão em ASCII, mas o Unicode de 16 bits (usado em Java) está ganhando popularidade com a internacionalização dos computadores. Até o início da década de 1980, a maioria dos fabricantes de computador escolhia sua própria representação de ponto flutuante. Quase todos os computadores a partir daquela época seguiram o mesmo padrão para ponto flutuante, o padrão IEEE 754. O padrão de ponto flutuante IEEE será discutido em detalhes no Apêndice J.

Algumas arquiteturas fornecem operações em strings de caractere, embora essas operações normalmente sejam muito limitadas e tratem cada byte na string como um único caractere. Operações típicas baseadas em strings de caractere são comparações e movimentações.

Para aplicações empresariais, algumas arquiteturas aceitam um formato decimal, normalmente chamado de *decimal compacto* ou *decimal codificado em binário (código BCD)* — 4 bits são usados para codificar os valores 0-9 e dois dígitos decimais são compactados em cada byte. As strings de caractere numérico às vezes são chamadas *decimal não compactado*, e as operações — chamadas *compactação* ou *descompactação* — normalmente são fornecidas para a conversão entre elas.

Uma razão para usar operandos decimais é obter resultados que coincidam exatamente com números decimais, já que algumas frações decimais não têm uma representação exata em binário. Por exemplo, $0,10_{10}$ é uma fração simples em decimal, mas, em binário, ela requer uma quantidade infinita de dígitos repetitivos: $0,00011\ 0011\ \dots_2$. Assim, cálculos que são exatos em decimal podem estar próximos mas imprecisos em binário, o que pode ser um problema para transações financeiras (veja mais sobre aritmética de precisão no Apêndice J).

Nossos benchmarks SPEC usam tipos de dados de byte ou caractere, meia palavra (inteiro curto), palavra (inteiro), palavra dupla (inteiro longo) e ponto flutuante. A [Figura A.11](#) mostra a distribuição dinâmica dos tamanhos dos objetos referenciados pela memória para esses programas. A frequência de acesso a tipos de dados diferentes ajuda a decidir quais os tipos mais importantes para se ter um suporte eficiente. O computador deve ter um datapath de 64 bits ou seria satisfatório dois ciclos para acessar uma palavra dupla? Como vimos anteriormente, os acessos de byte requerem uma rede de alinhamento: quanto importante é aceitar bytes como primitivos? A [Figura A.11](#) usa referências de memória para examinar os tipos de dados que estão sendo acessados.

Em algumas arquiteturas, objetos em registradores podem ser acessados como bytes ou meias palavras. Porém, tal acesso é muito raro — no VAX, isso representa não mais

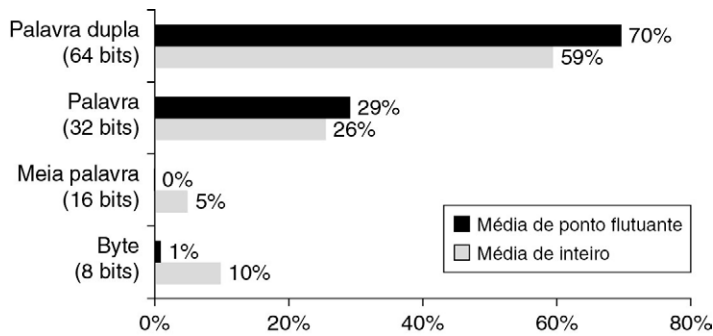


FIGURA A.11 Distribuição de acessos a dados por tamanho para os programas de benchmark.

O tipo de dados palavra dupla é usado para ponto flutuante de dupla precisão em programas de ponto flutuante e para endereços, desde que o computador use endereços de 64 bits. Em um computador de endereço de 32 bits, os endereços de 64 bits seriam substituídos por endereços de 32 bits e, portanto, quase todos os endereços de palavra dupla nos programas de inteiro se tornariam endereços de palavra simples.

de 12% das referências de registradores ou aproximadamente 6% de todos os acessos de operandos nesses programas.

A.5 OPERAÇÕES NO CONJUNTO DE INSTRUÇÕES

Os operadores aceitos pela maioria das arquiteturas de conjunto de instruções podem ser categorizados como na [Figura A.12](#). Uma regra geral para todas as arquiteturas é que as instruções mais frequentemente executadas são as operações simples de um conjunto de instruções. Por exemplo, a [Figura A.13](#) mostra 10 instruções simples que respondem por

Tipo de operador	Exemplos
Aritmético e lógico	Operações aritméticas e lógicas de inteiros: adição, subtração e (and), ou (or), multiplicação, divisão
Transferência de dados	Carregamentos-armazenamentos (instruções de movimentação em computadores com endereçamento de memória)
Controle	Desvio, salto, chamada e retorno de procedimento, traps
Sistema	Chamada de sistema operacional, instruções de gerenciamento de memória virtual
Ponto flutuante	Operações de ponto flutuante: adição, multiplicação, divisão, comparação
Decimal	Adição decimal, multiplicação decimal, conversão decimal para caracteres
String	Movimento de string, comparação de string, pesquisa de string
Gráficos	Operações de pixels e vértices, operações de compactação/descompactação

FIGURA A.12 Categorias dos operadores de instrução e exemplos de cada uma.

Todos os computadores normalmente fornecem um conjunto completo de operações para as três primeiras categorias. O suporte para funções de sistema no conjunto de instruções varia amplamente entre as arquiteturas, mas todos os computadores precisam ter algum suporte de instrução para as funções básicas do sistema. A quantidade de suporte no conjunto de instruções para as quatro últimas categorias pode variar de nenhum até um extenso conjunto de instruções especiais. As instruções de ponto flutuante serão fornecidas em qualquer computador destinado a uma aplicação que utiliza intensamente ponto flutuante. Algumas vezes, essas instruções são parte de um conjunto de instruções opcional. As instruções de decimais e string são, às vezes, primitivas, como no VAX ou IBM 360, ou podem ser sintetizadas pelo compilador a partir de instruções mais simples. As instruções gráficas normalmente operam em muitos itens de dados menores em paralelo, por exemplo, realizando oito adições de 8 bits em operandos de 64 bits.

Classificação	Instrução do 80x86	Média de inteiro (% total executada)
1	load	22%
2	desvio condicional	20%
3	compare	16%
4	store	12%
5	add	8%
6	and	6%
7	sub	5%
8	move registrador-registrador	4%
9	call	1%
10	return	1%
Total		96%

FIGURA A.13 As 10 instruções principais para o 80x86.

As instruções simples dominam essa lista e são responsáveis por 96% das instruções executadas. Essas porcentagens são a média dos cinco programas SPECint92.

96% das instruções executadas para uma coleção de programas de inteiro sendo executados no popular Intel 80x86. Consequentemente, o implementador dessas instruções deve cuidar para torná-las rápidas, já que elas são o caso comum.

Como já dissemos, as instruções na [Figura A.13](#) são encontradas em todo computador para qualquer aplicação — desktop, servidor, embarcado — com as variações das operações na [Figura A.12](#) dependendo fortemente dos tipos de dados tratados pelo conjunto de instruções.

A.6 INSTRUÇÕES PARA FLUXO DE CONTROLE

Como as medidas de comportamento de desvios e saltos são completamente independentes das outras medidas e aplicações, examinamos agora o uso das instruções de fluxo de controle, que têm pouco em comum com as operações das seções anteriores.

Não há uma terminologia consistente para instruções que mudam o fluxo de controle. Na década de 1950, elas normalmente eram chamadas *transferências*. A partir de 1960, o nome *desvio* começou a ser usado. Mais tarde, os computadores introduziram nomes adicionais. Em todo este livro, usamos *salto* quando a mudança no controle for incondicional e *desvio* quando a mudança for condicional.

Podemos distinguir quatro tipos diferentes de mudança de fluxo de controle:

- Desvios condicionais
- Saltos
- Chamadas de procedimento
- Retornos de procedimento

Queremos saber a frequência relativa desses eventos, já que cada evento é diferente, pode usar instruções diferentes e ter comportamento diferente. [Figura A.14](#) mostra as frequências dessas instruções de fluxo de controle para um computador de carregamento-armazenamento executando nossos benchmarks.

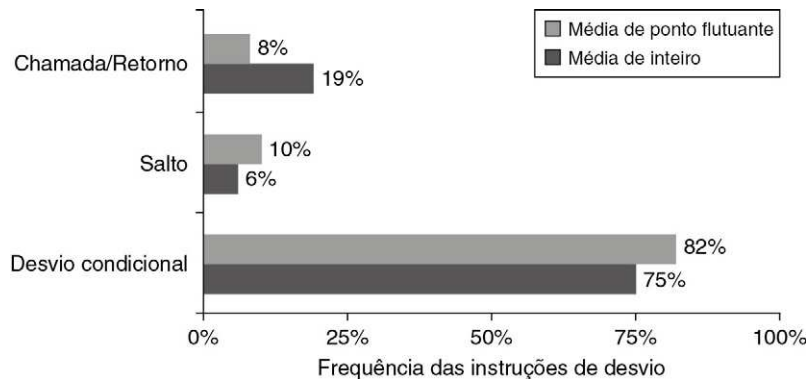


FIGURA A.14 As instruções de fluxo de controle podem ser divididas em três classes: chamadas ou retornos, saltos e desvios condicionais.

Os desvios condicionais dominam claramente. Cada tipo é contado em uma de três barras. Os programas e o computador usados para coletar essas estatísticas são os mesmos da [Figura A.8](#).

Modos de endereçamento para instruções de fluxo de controle

O endereço de destino de uma instrução de fluxo de controle sempre deve ser especificado. Esse destino é especificado explicitamente na instrução, na grande maioria dos casos — sendo o retorno de procedimento a principal exceção, já que, para o retorno, o destino não é conhecido em tempo de compilação. O modo mais comum de especificar o destino é fornecer um deslocamento que seja acrescentado ao *contador de programa* (PC). As instruções de fluxo de controle desse tipo são chamadas *relativas ao PC*. Os desvios ou saltos relativos ao PC são vantajosos porque, frequentemente, o destino está próximo à instrução atual, e especificar a posição relativa ao PC atual requer menos bits. Usar o endereçamento relativo ao PC também permite que o código seja executado independentemente de onde esteja carregado. Essa propriedade, chamada *independência de posição*, pode eliminar algum trabalho quando o programa é linkeditado e também é útil em programas linkeditados dinamicamente durante a execução.

Para implementar retornos e saltos indiretos quando o destino não é conhecido em tempo de compilação, é necessário um método diferente do endereçamento relativo ao PC. Aqui deve haver uma maneira de especificar o destino dinamicamente, para que ele possa mudar em tempo de execução. Esse endereço dinâmico pode ser tão simples quanto nomear um registrador que contém o endereço de destino; alternativamente, o salto pode permitir que qualquer modo de endereçamento seja usado para fornecer o endereço de destino.

Esses saltos indiretos de registrador também são úteis para outros quatro recursos importantes:

- Instruções *case* ou *switch*, encontradas na maioria das linguagens de programação (que selecionam uma entre várias alternativas).
- *Funções* ou *métodos virtuais* em linguagens orientadas a objeto, como C++ ou Java (que permitem que diferentes rotinas sejam chamadas dependendo do tipo de argumento).
- *Funções de ordem superior* ou *ponteiros de função* em linguagens como C ou C++ (que permitem que funções sejam passadas como argumentos, dando um pouco do “sabor” da programação orientada a objeto).
- *Bibliotecas compartilhadas dinamicamente* (que permitem carregar e linkeditar uma biblioteca em tempo de execução apenas quando é efetivamente chamada pelo programa em vez de carregada e linkeditada estaticamente antes de o programa ser executado).

Nos quatro casos, o endereço de destino não é conhecido em tempo de compilação e, conseqüentemente, costuma ser carregado da memória em um registrador antes do salto indireto de registrador.

Como os desvios geralmente usam endereçamento relativo ao PC para especificar seus destinos, uma importante questão é quanto os destinos de desvio estão distantes dos desvios. Saber a distribuição desses deslocamentos ajudará na escolha dos deslocamentos (offsets) de desvio que devem ser suportados e, assim, afetará o tamanho das instruções e a codificação. A [Figura A.15](#) mostra a distribuição dos deslocamentos para desvios relativos ao PC nas instruções. Aproximadamente 75% dos desvios estão voltados para a frente.

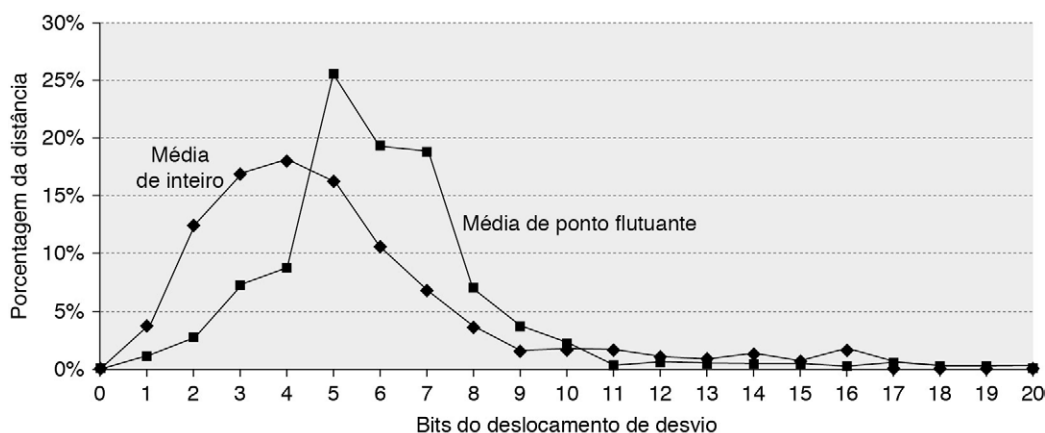


FIGURA A.15 Distâncias de desvio em termos do número de instruções entre o destino e a instrução de desvio.

Os desvios mais frequentes nos programas de inteiro são para destinos que podem ser codificados em 4-8 bits. Esse resultado nos diz que campos de deslocamento curtos normalmente são suficientes para desvios e que o projetista pode ganhar alguma densidade de codificação tendo uma instrução mais curta com um deslocamento de desvio menor. Essas medições foram feitas em um computador de carregamento-armazenamento (arquitetura Alpha) com todas as instruções alinhadas nos limites da palavra. Uma arquitetura que exige menos instruções para o mesmo programa, como um VAX, teria distâncias de desvio mais curtas. Entretanto, o número de bits necessários para o deslocamento pode aumentar se o computador tiver instruções de tamanho variável para serem alinhadas em qualquer limite de byte. Os programas e o computador usados para coletar essas estatísticas são os mesmos da [Figura A.8](#).

Opções do desvio condicional

Como a maioria das mudanças no fluxo ocorre em desvios, é importante decidir como especificar a condição do desvio. A [Figura A.16](#) mostra as três principais técnicas em uso atualmente e suas vantagens e desvantagens.

Uma das propriedades mais notáveis dos desvios é que um grande número de comparações consiste em testes simples e outro grande número em comparações com zero. Assim, algumas arquiteturas escolhem tratar essas comparações como casos especiais, sobretudo se uma instrução *compare e desvio* estiver sendo usada. A [Figura A.17](#) mostra a frequência das diferentes comparações usadas para desvio condicional.

Opções de chamada de procedimento

Chamadas e retornos de procedimento incluem transferência de controle e, possivelmente, a gravação de algum estado; no mínimo, o endereço de retorno deve ser salvo em algum lugar, às vezes em um registrador de link especial ou simplesmente em um GPR. Algumas

Nome	Exemplos	Como a condição é testada	Vantagens	Desvantagens
Código de condição (CC)	80x86, ARM, PowerPC, SPARC, SuperH	Testa bits especiais definidos pelas operações de ULA, possivelmente sob o controle do programa.	Algumas vezes, a condição é definida sem custo.	CC é um estado extra. Os códigos de condição restringem a ordenação das instruções, já que eles passam informações de uma instrução para um desvio.
Registrador de condição	Alpha, MIPS	Testa um registrador arbitrário com o resultado de uma comparação.	Simples.	Consome um registrador.
Comparação e desvio	PA-RISC, VAX	A comparação é parte do desvio. Muitas vezes, a comparação é limitada a um subconjunto.	Uma instrução em vez de trabalho por duas para um desvio.	Pode envolver muito trabalho por instrução para execução em pipeline.

FIGURA A.16 Os principais métodos para avaliar condições de desvio, suas vantagens e suas desvantagens.

Embora os códigos de condição possam ser definidos por operações da ULA necessárias para outros fins, as medições nos programas mostram que isso raramente acontece. Os principais problemas de implementação com códigos de condição surgem quando o código de condição é definido por um subconjunto de instruções grande ou aleatoriamente escolhido, em vez de ser controlado por um bit na instrução. Os computadores com comparação e desvio normalmente limitam o conjunto de comparações e usam um registrador de condição para comparações mais complexas. Muitas vezes, técnicas diferentes são usadas para desvios com base em comparação de ponto flutuante em vez daquelas baseadas em comparação de inteiro. Essa dicotomia faz sentido, já que o número de desvios que dependem das comparações de ponto flutuante é muito menor do que o número dependendo das comparações de inteiro.

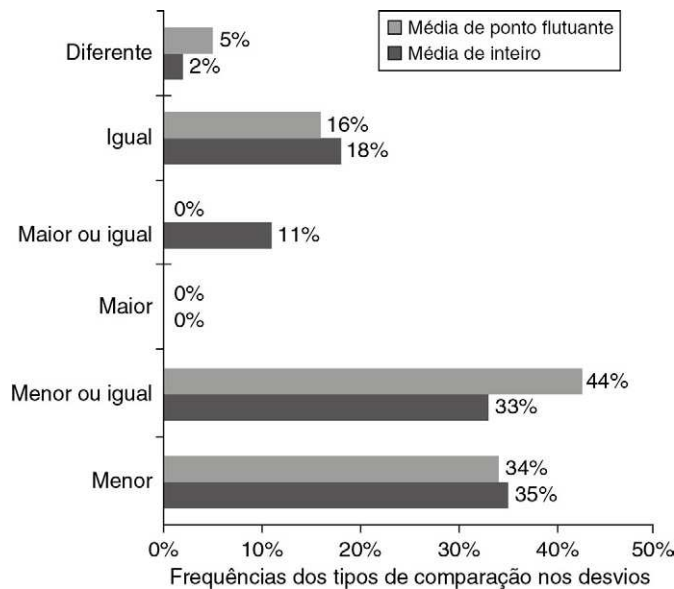


FIGURA A.17 Frequência de diferentes tipos de comparações em desvios condicionais.

Desvios menores (ou iguais) dominam essa combinação de compilador e arquitetura. Essas medições incluem as comparações de inteiros e ponto flutuante nos desvios. Os programas e o computador utilizados para coletar essas estatísticas são iguais aos da [Figura A.8](#).

arquiteturas mais antigas fornecem um mecanismo para salvar muitos registradores, enquanto arquiteturas mais novas exigem que o compilador gere operações de armazenamento e carregamentos para cada registrador salvo e restaurado.

Há duas convenções básicas em uso para salvar registradores: ou no local de chamada ou dentro do procedimento que está sendo chamado (invocado). *Salvamento por quem chama* significa que o procedimento que chama precisa salvar os registradores que deseja

preservar para o acesso depois da chamada; assim, o procedimento chamado não precisa se preocupar com registradores. *Salvamento pelo chamado* é o contrário: o procedimento chamado precisa salvar os registradores que deseja usar, deixando quem chama livre. Há ocasiões em que o salvamento de quem chama precisa ser usado devido a padrões de acesso a variáveis visíveis globalmente em dois procedimentos diferentes. Por exemplo, suponha que tenhamos um procedimento P1 que chama o procedimento P2 e ambos os procedimentos manipulem a variável global x . Se P1 tiver alocado x para um registrador, ele deve se certificar em salvar x em um local conhecido por P2 antes da chamada a P2. A habilidade de um compilador para descobrir quando um procedimento chamado pode acessar quantidades alocadas em registradores é complicada pela possibilidade da compilação separada. Suponha que P2 não possa acessar x mas possa chamar outro procedimento, P3, que pode acessar x , embora P2 e P3 sejam compilados separadamente. Devido a essas complicações, a maioria dos compiladores salvará de modo conservador *qualquer* variável que possa ser acessada durante uma chamada.

Nos casos em que qualquer convenção poderia ser usada, alguns programas serão mais eficientes com salvamento pelo chamado e outros serão mais eficientes com salvamento pelo chamador. Como resultado, a maioria dos sistemas reais de hoje usa uma combinação dos dois mecanismos. Essa convenção é especificada em uma interface de aplicação binária (ABI) que estabelece as regras básicas quanto aos registradores que devem ser salvos pelo chamador e quais devem ser salvos pelo chamado. Mais adiante, neste apêndice, examinaremos a divergência entre instruções sofisticadas para salvar automaticamente registradores e as necessidades do compilador.

Resumo: instruções para fluxo de controle

As instruções de fluxo de controle são algumas das instruções mais executadas. Embora haja muitas opções para desvios condicionais, esperávamos que o endereçamento de desvio em uma nova arquitetura fosse capaz de saltar centenas de instruções acima ou abaixo do desvio. Essa exigência sugere um deslocamento de desvio relativo ao PC de pelo menos 8 bits. Esperávamos, também, ver o endereçamento indireto de registrador e o relativo ao PC para instruções de salto, a fim de aceitar retornos, bem como muitos outros recursos dos sistemas atuais.

Agora completamos nosso “passeio” pela arquitetura da instrução no nível visto por um programador de linguagem assembly ou um projetista de compilador. Estamos nos voltando para uma arquitetura de carregamento-armazenamento com os modos de endereçamento de deslocamento, imediato e indireto de registrador. Esses dados são inteiros de 8, 16, 32 e 64 bits e dados de ponto flutuante de 32 e 64 bits. As instruções incluem operações simples, desvios condicionais relativos ao PC, instruções de salto e link para chamada de procedimento e saltos indiretos para retorno de procedimento (além de alguns outros usos).

Agora, precisamos escolher como representar essa arquitetura de uma forma que facilite a execução pelo hardware.

A.7 CODIFICAÇÃO DE UM CONJUNTO DE INSTRUÇÕES

Obviamente, as escolhas mencionadas anteriormente afetarão o modo como as instruções são codificadas em uma representação binária para execução pelo processador. Essa representação afeta não só o tamanho do programa compilado, mas também a implementação do processador, que precisa decodificá-la para encontrar rapidamente a operação e seus operandos. A operação normalmente é especificada em um campo

Operação e número de operandos	Especificador de endereço 1	Campo de endereço 1	...	Especificador de endereço n	Campo de endereço n
--------------------------------	-----------------------------	---------------------	-----	-------------------------------	-----------------------

(a) Variável (p. ex., Intel 80x86, VAX)

Operação	Campo de endereço 1	Campo de endereço 2	Campo de endereço 3
----------	---------------------	---------------------	---------------------

(b) Fixo (p. ex., Alpha, ARM, MIPS, PowerPC, SPARC, SuperH)

Operação	Especificador de endereço	Campo de endereço
----------	---------------------------	-------------------

Operação	Especificador de endereço 1	Especificador de endereço 2	Campo de endereço
----------	-----------------------------	-----------------------------	-------------------

Operação	Especificador de endereço	Campo de endereço 1	Campo de endereço 2
----------	---------------------------	---------------------	---------------------

(c) Híbrido (p. ex., IBM 360/370, MIPS16, Thumb, TI TMS320C54x)

FIGURA A.18 Três variações básicas na codificação de instrução. Tamanho variável, tamanho fixo e híbrido.

O formato variável pode aceitar qualquer número de operandos, com cada especificador de endereço determinando o modo de endereçamento e o tamanho do especificador para esse operando. Ele geralmente permite a menor representação de código, já que os campos não usados não precisam ser incluídos. O formato fixo sempre tem o mesmo número de operandos, com os modos de endereçamento (se houver opções) especificados como parte do opcode. Normalmente, ele resulta no tamanho de código maior. Embora os campos não costumem variar em seu local, eles serão usados para diferentes finalidades por diferentes instruções. O método híbrido possui vários formatos especificados pelo opcode, acrescentando um ou dois campos para especificar o modo de endereçamento e um ou dois campos para especificar o endereço do operando.

chamado *opcode*. Como veremos, uma decisão importante é como codificar os modos de endereçamento com as operações.

Essa decisão depende da faixa dos modos de endereçamento e do grau de independência entre opcodes e modos. Alguns computadores mais antigos possuem 1-5 operandos com 10 modos de endereçamento para cada operando (Fig. A.6). Para um número tão grande de combinações, normalmente um *especificador de endereço* separado é necessário para cada operando: o especificador de endereço informa que modo de endereçamento é usado para acessar o operando. No outro extremo estão computadores de carregamento-armazenamento com um único operando de memória e apenas um ou dois modos de endereçamento; obviamente, nesse caso, o modo de endereçamento pode ser codificado como parte do opcode.

Ao codificar as instruções, o número de registradores e o número de modos de endereçamento têm um impacto significativo no tamanho das instruções, já que o campo de registrador e o campo de modo de endereçamento podem aparecer muitas vezes em uma única instrução. Na realidade, para a maioria das instruções, muito mais bits são consumidos nos campos modo de endereçamento e de registrador do que na especificação do opcode. O arquiteto precisa equilibrar várias forças concorrentes ao codificar o conjunto de instruções:

1. O desejo de ter o máximo de registradores e modos de endereçamento possíveis.
2. O impacto do tamanho dos campos de registrador e do modo de endereçamento no tamanho médio da instrução e, conseqüentemente, no tamanho médio do programa.
3. O desejo de ter instruções codificadas em tamanhos que sejam fáceis de controlar em uma implementação com pipeline (a importância das instruções facilmente

decodificadas será discutida no Apêndice C e no Capítulo 3). Como um mínimo, o arquiteto quer que as instruções sejam em múltiplos de bytes, e não em um comprimento em bits arbitrário. Muitos arquitetos de computadores desktop e de servidores escolhem usar uma instrução de tamanho fixo para ganhar vantagens de implementação enquanto sacrificam o tamanho médio do código.

A Figura A.18 mostra três escolhas comuns para codificar o conjunto de instruções. À primeira chamamos *variável*, já que permite que quase todos os modos de endereçamento estejam com todas as operações. Esse estilo é melhor quando há muitos modos de endereçamento e operações. À segunda opção, chamamos *fixa*, uma vez que combina a operação e o modo de endereçamento no opcode. Frequentemente, a codificação fixa terá um único tamanho para todas as instruções; ela funciona melhor quando há poucos modos de endereçamento e operações. O compromisso entre codificação variável e codificação fixa é o do tamanho dos programas contra a facilidade de decodificação no processador. A variável tenta usar o mínimo possível de bits para representar o programa, mas as instruções individuais podem variar amplamente em tamanho e na quantidade de trabalho a ser executado.

Vejamos uma instrução do 80x86 para ter um exemplo da codificação variável:

```
add EAX, 1000(EBX)
```

O nome `add` significa uma instrução `add` de inteiro de 32 bits com dois operandos, e esse opcode leva 1 byte. Um especificador de endereço do 80x86 é de 1 ou 2 bytes, especificando o registrador origem/destino (EAX) e o modo de endereçamento (deslocamento nesse caso) e o registrador-base (EBX) para o segundo operando. Essa combinação leva 1 byte para especificar os operandos. Quando no modo de 32 bits (Apêndice K), o tamanho do campo endereço é 1 byte ou 4 bytes. Como 1.000 é maior que 2^8 , o tamanho total da instrução é

$$1 + 1 + 4 = 6 \text{ bytes}$$

O tamanho das instruções 80x86 varia entre 1-17 bytes. Os programas 80x86 normalmente são maiores do que as arquiteturas RISC, que usam formatos fixos (Apêndice K).

Dados esses dois polos do projeto do conjunto de instruções da variável e fixa, a terceira alternativa imediatamente vem à mente: reduzir a variabilidade em tamanho e trabalho da arquitetura variável, mas fornecer vários tamanhos de instrução para reduzir o tamanho do código. Esse método *híbrido* é a terceira alternativa de codificação, e veremos exemplos em breve.

Tamanho de código reduzido nos RISCs

Quando os computadores RISC começaram a ser usados em aplicações embarcadas, o formato fixo de 32 bits tornou-se um problema, pois o custo e, consequentemente, o tamanho menor do código são importantes. Em resposta, vários fabricantes ofereceram uma nova versão híbrida de seus conjuntos de instruções RISC, com instruções de 16 bits e 32 bits. As instruções curtas aceitam menos operações, menos campos de endereços e campos imediatos, menor quantidade de registradores e um formato de dois endereços em vez do clássico formato de três endereços dos computadores RISC. O Apêndice K fornece dois exemplos, o ARM Thumb e o MIPS16 do MIPS, que ostentam uma redução no tamanho do código de até 40%.

Em contraste com essas extensões de conjunto de instruções, a IBM simplesmente compacta seu conjunto de instruções-padrão e, depois, acrescenta hardware para descompactar ins-

truções, como se elas fossem buscadas da memória em uma falta na cache (miss cache) de instruções. Portanto, a cache de instruções contém instruções de 32 bits completas, mas o código compactado é mantido na memória principal, em ROM e no disco. A vantagem do MIPS16 e do Thumb é que as caches de instrução agem como se fossem aproximadamente 25% maiores, enquanto o CodePack da IBM significa que os compiladores não precisam ser alterados para controlar diferentes conjuntos de instruções, e a decodificação da instrução pode permanecer simples.

O CodePack começa com compactação de codificação run-length em qualquer programa PowerPC e, depois, carrega as tabelas de compactação resultantes em uma tabela de 2 KB no chip. Consequentemente, todo programa tem sua própria codificação única. Para manipular desvios, que não são mais para um limite de palavra, o PowerPC cria uma tabela de hash na memória que faz o mapeamento entre endereços compactados e descompactados. Como um TLB (Cap. 2), ele coloca na cache os mapas de endereço mais recentemente usados, para reduzir o número de acessos à memória. A IBM alega um custo de desempenho geral de 10%, resultando em uma redução de tamanho de código de 35-40%.

A Hitachi simplesmente inventou um conjunto de instruções RISC com formato fixo de 16 bits, chamado SuperH, para aplicações embarcadas (Apêndice H). Ele possui 16 em vez de 32 registradores, para fazê-lo caber no formato mais estreito e com menos instruções, mas, fora isso, se parece com uma arquitetura RISC clássica.

Resumo: codificando um conjunto de instruções

As decisões tomadas nos componentes do projeto de conjunto de instruções, discutidas nas seções anteriores, determinam se o arquiteto tem a escolha entre codificações de instruções variáveis e fixas. Dada a escolha, o arquiteto mais interessado no tamanho de código do que no desempenho escolherá a codificação variável, e o mais interessado no desempenho do que no tamanho de código escolherá a codificação fixa. O Apêndice E traz 13 exemplos dos resultados das escolhas de arquitetos. No Apêndice C e no Capítulo 3, o impacto da variabilidade sobre o desempenho do processador será discutido em detalhes.

Estamos quase terminando de preparar a base para a arquitetura de conjunto de instruções MIPS que será introduzida na [Seção A.9](#). Antes de disso, porém, será útil dar uma olhada na tecnologia do compilador e seu efeito nas propriedades do programa.

A.8 QUESTÕES GERAIS: O PAPEL DOS COMPILADORES

Hoje, quase toda programação é feita em linguagens de alto nível para aplicações de desktops e servidores. Esse desenvolvimento significa que, como a maioria das instruções executadas é a saída de um compilador, uma arquitetura de conjunto de instruções é essencialmente o destino de um compilador. Antigamente, para essas aplicações, muitas vezes eram tomadas decisões arquitetônicas para facilitar a programação em linguagem assembly ou para um kernel específico. Como o compilador afetará significativamente o desempenho de um computador, entender a tecnologia de compilador moderna é vital para projetar e implementar eficientemente um conjunto de instruções.

Já foi comum tentar isolar a tecnologia de compiladores e seu efeito sobre o desempenho de hardware, da arquitetura e seu desempenho, da mesma maneira que era comum tentar separar a arquitetura da sua implementação. Essa separação é praticamente impossível com os compiladores e computadores desktops atuais. As escolhas arquitetônicas afetam positiva ou negativamente a qualidade do código que pode ser gerado para um computador e a complexidade de construir um bom compilador para ele.

Nesta seção, discutiremos os principais objetivos no conjunto de instruções, sobretudo do ponto de vista do compilador. Ela começa com uma revisão da anatomia dos compiladores atuais. Em seguida, discutiremos como a tecnologia do compilador afeta as decisões do arquiteto e como o arquiteto pode dificultar ou facilitar a produção de um bom código pelo compilador. Concluiremos com uma revisão dos compiladores e operações de multimídia, o que, infelizmente, é um mau exemplo de cooperação entre os projetistas de compilador e arquitetos.

A estrutura dos compiladores modernos

Para começar, vejamos como são os compiladores de otimização de hoje. A [Figura A.19](#) mostra a estrutura de compiladores recentes.

O primeiro objetivo de um projetista de compilador é a correção — todos os programas válidos precisam ser compilados corretamente. O segundo objetivo normalmente é a velocidade do código compilado. Em geral, todo um conjunto de outros objetivos segue esses dois, incluindo compilação rápida, suporte à depuração e interoperabilidade entre linguagens. Normalmente, os passos no compilador transformam as representações de alto nível e mais abstratas em representações de nível progressivamente mais baixos. Eventualmente, ele atinge o conjunto de instruções. Essa estrutura ajuda a gerenciar a complexidade das transformações e facilita a escrever um compilador livre de bugs.

A complexidade de escrever um compilador correto é uma grande limitação sobre a quantidade de otimização que pode ser feita. Embora a estrutura de passos múltiplos ajude a reduzir a complexidade do compilador, isso também significa que o compilador precisa ordenar e realizar algumas transformações antes de outras. No diagrama do compilador otimizado na [Figura A.19](#), podemos ver que certas otimizações de alto nível são realizadas

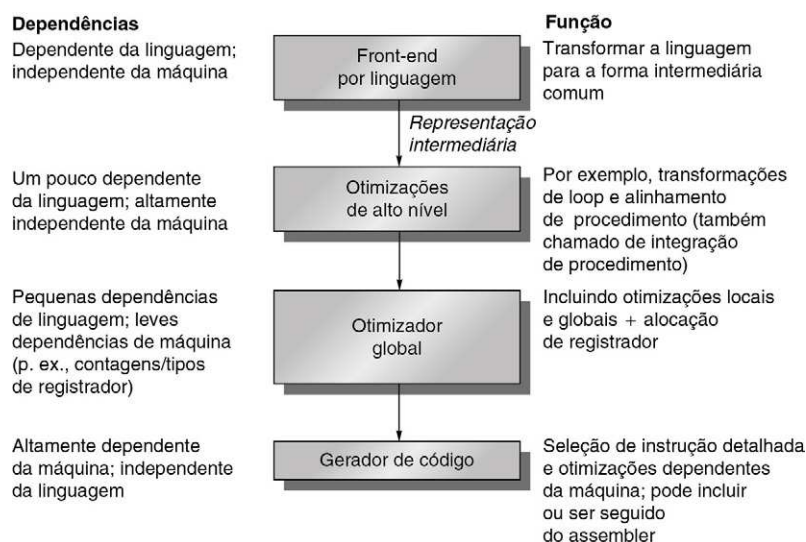


FIGURA A.19 Os compiladores normalmente consistem em 2-4 passos, com os compiladores de otimização mais alta tendo mais passos.

Essa estrutura maximiza a probabilidade de um programa compilado em vários níveis de otimização produzir a mesma saída quando recebe a mesma entrada. Os passos de otimização se destinam a ser opcionais e podem ser ignorados quando o objetivo é a compilação mais rápida e o código de qualidade inferior é aceitável. Um passo é simplesmente uma fase em que o compilador lê e transforma o programa inteiro. (O termo *fase* normalmente é usado de maneira indistinta de *passo*.) Como os passos são separados, várias linguagens podem usar os mesmos passos de otimização e de geração código. Apenas um novo front-end é necessário para uma nova linguagem.

muito antes de se saber como será o código resultante. Quando essa transformação é feita, o compilador não pode deixar de voltar e visitar todas as etapas, possivelmente desfazendo transformações. Essa interação seria proibitiva, tanto em tempo de compilação quanto em complexidade. Portanto, os compiladores fazem suposições sobre a capacidade de etapas futuras lidarem com certos problemas. Por exemplo, os compiladores normalmente precisam escolher quais chamadas de procedimento expandir alinhadas antes de saberem o tamanho exato do procedimento que está sendo chamado. Os projetistas de compilador chamam esse problema de *problema de ordenação de fases*.

Como essa ordenação de transformações interage com a arquitetura do conjunto de instruções? Um bom exemplo ocorre com a otimização chamada *eliminação global de subexpressão comum*. Essa otimização encontra duas instâncias de uma expressão que calculam o mesmo valor e salvam o valor do primeiro cálculo em um temporário. Depois, ela usa o valor temporário, eliminando a segunda computação da expressão comum.

Para que essa otimização seja significativa, o temporário deve ser alocado em um registrador. Caso contrário, o custo de armazenar o temporário na memória e depois recarregá-lo pode minimizar as economias obtidas por não recalcular a expressão. Na verdade, há casos em que essa otimização efetivamente torna o código mais lento, quando o temporário não é alocado em um registrador. A ordenação de fase complica esse problema, porque a alocação de registradores normalmente é feita próximo do fim da passagem de otimização global, imediatamente antes da geração do código. Assim, um otimizador que execute essa otimização *deve* supor que o alocador de registradores vai alocar o temporário em um registrador.

As otimizações executadas por compiladores modernos podem ser classificadas pelo estilo da transformação, como segue:

- As *otimizações de alto nível* normalmente são feitas no código-fonte, com a saída como entrada de passos de otimização posteriores.
- As *otimizações locais* só otimizam código dentro de um fragmento de código em linha reta (chamado *bloco básico* pelo pessoal de compiladores).
- As *otimizações globais* estendem as otimizações locais através de desvios e introduzem um conjunto de transformações que visam à otimização de loops.
- A *alocação de registradores* associa registradores com operandos.
- As *otimizações dependentes de processador* tentam tirar proveito de conhecimento de arquiteturas específicas.

Alocação de registradores

Devido ao importante papel desempenhado pela alocação de registradores, tanto em acelerar o código quanto em tornar outras otimizações úteis, ela é uma das mais importantes — se não a mais importante — das otimizações. Os algoritmos de alocação de registradores atuais são baseados em uma técnica chamada *coloração de grafo*. A ideia básica por trás da coloração de grafo é construir um grafo que represente os possíveis candidatos para alocação a um registrador e, depois, usar o grafo para alocar registradores. Geralmente, o problema é como usar um conjunto limitado de cores de modo que nenhum par de nós adjacentes em um grafo de dependência tenha a mesma cor. A ênfase do método é alcançar 100% de alocação das variáveis ativas a registradores. O problema de colorir um grafo em geral pode levar um tempo exponencial em função do tamanho do grafo (NP completo). Entretanto, existem algoritmos heurísticos que funcionam bem na prática, produzindo alocações aproximadas que são executadas em tempo quase linear.

A coloração de grafo funciona melhor quando há pelo menos 16 (e preferivelmente mais) registradores de propósito geral disponíveis para alocação global de variáveis inteiras e

registradores adicionais para ponto flutuante. Infelizmente, a coloração de grafos não funciona muito bem quando o número de registradores é pequeno, porque os algoritmos heurísticos para colorir o grafo provavelmente falharão.

Impacto das otimizações no desempenho

Às vezes é difícil separar algumas das otimizações mais simples — otimizações locais e dependentes do processador — das transformações feitas no gerador de código. Exemplos de otimizações típicas são fornecidos na [Figura A.20](#). A última coluna da [Figura A.20](#) indica a frequência com que as transformações de otimização listadas foram aplicadas ao programa de origem.

A [Figura A.21](#) mostra o efeito de várias otimizações em instruções executadas para dois programas. Nesse caso, os programas otimizados executaram aproximadamente 25-90%

Nome da otimização	Explicação	Porcentagem do número total das transformações otimizadoras
<i>Alto nível</i>	<i>No nível de código-fonte ou próximo dele; independente do processador</i>	
Integração de procedimento	Substitui a chamada de procedimento pelo corpo do procedimento	N.M.
<i>Local</i>	<i>Dentro do código em linha reta</i>	
Eliminação da subexpressão comum	Substitui duas instâncias da mesma computação por uma única cópia	18%
Propagação constante	Substitui todas as instâncias de uma variável a que é atribuída uma constante pela constante	22%
Redução da altura da pilha	Rearranja a árvore de expressão para minimizar recursos necessários para a avaliação da expressão	N.M.
<i>Global</i>	<i>Através de um desvio</i>	
Eliminação global da subexpressão comum	O mesmo que a eliminação global, mas essa versão estende a todos os desvios	13%
Propagação de cópia	Substitui todas as instâncias de uma variável A a que foi atribuído X (p. ex., $A = X$) por X	11%
Movimento de código	Remove código de um loop que calcula algum valor a cada iteração do loop	16%
Eliminação da variável de indução	Simplifica/elimina os cálculos de endereçamento de array dentro de loops	2%
<i>Dependente do processador</i>	<i>Depende do conhecimento do processador</i>	
Redução de força	Muitos exemplos, como substituir multiplicação por uma sequência de adições e deslocamentos	N.M.
Programação de pipeline	Reordena instruções para melhorar o desempenho do pipeline	N.M.
Otimização do deslocamento de desvio	Escolhe o menor deslocamento de desvio que atinja o destino	N.M.

FIGURA A.20 Principais tipos de otimizações e exemplos em cada classe.

Esses dados informam sobre a frequência relativa da ocorrência das várias otimizações. A terceira coluna lista a frequência estática com a qual algumas otimizações comuns são aplicadas em um conjunto de 12 pequenos programas FORTRAN e Pascal. Existem nove otimizações locais e globais feitas pelo compilador incluído na medição. Seis dessas otimizações são cobertas na figura e as outras três respondem por 18% das ocorrências estáticas totais. A abreviatura N.M. significa que o número de ocorrências dessa otimização não foi medido. As otimizações dependentes de processador geralmente são feitas em um gerador de código, e nenhuma delas foi medida nessa experiência. A porcentagem é a parte das otimizações estáticas que são do tipo especificado. *Dados de Chow, 1983 (coletados usando o compilador Stanford UCODE).*

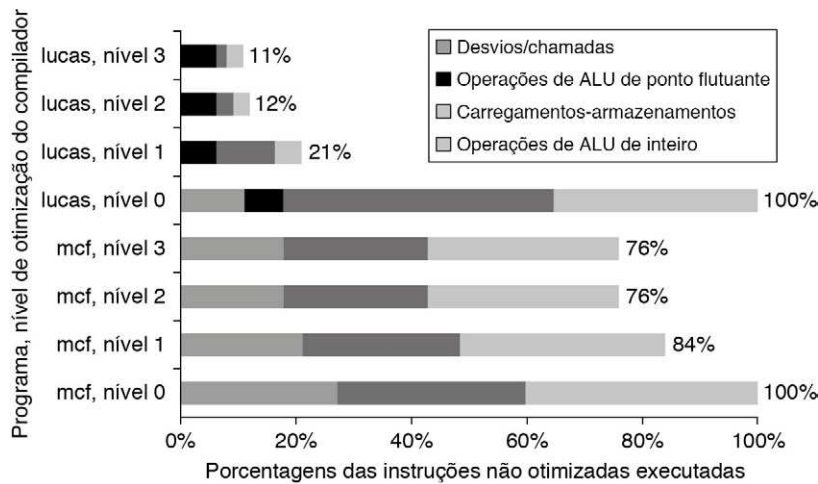


FIGURA A.21 Mudança na contagem de instrução para os programas lucas e mcf do SPEC2000 conforme a variação dos níveis de otimização do compilador.

O nível 0 é igual ao código não otimizado. O nível 1 inclui otimizações locais, programação de código e alocação de registrador local. O nível 2 inclui otimizações globais, transformações de loop (pipelining de software) e alocação de registrador global. O nível 3 acrescenta integração de procedimento. Essas experiências foram realizadas nos compiladores Alpha.

menos instruções do que programas não otimizados. A figura ilustra a importância de olhar o código otimizado antes de sugerir novos recursos de conjuntos de instruções, já que um compilador poderia remover completamente as instruções que o arquiteto estava tentando melhorar.

O impacto da tecnologia de compilador nas decisões do arquiteto

A interação entre os compiladores e as linguagens de alto nível afeta significativamente a maneira como os programas usam a arquitetura de conjunto de instruções. Há duas questões importantes: como as variáveis são alocadas e endereçadas; e quantos registradores são necessários para alocar as variáveis corretamente. Para tratar dessas questões, precisamos examinar três áreas separadas em que as linguagens de alto nível atuais alocam seus dados:

- A *pilha* é usada para alocar variáveis locais. A pilha aumenta ou diminui na chamada ou no retorno de um procedimento, respectivamente. Os objetos na pilha são endereçados em relação ao ponteiro da pilha e são principalmente escalares (variáveis simples) em vez de arrays. A pilha é usada para registros de ativação, *não* como uma pilha para avaliar expressões. Portanto, valores quase nunca são enviados ou recuperados na pilha.
- A *área de dados globais* é usada para alocar objetos estaticamente declarados, como variáveis globais e constantes. Grande parte desses objetos é de arrays e outras estruturas de dados agregadas.
- A *pilha heap* é usada para alocar objetos dinâmicos que não aderem a uma disciplina de pilha. Os objetos na pilha heap são acessados com ponteiros e normalmente não são escalares.

A alocação de registradores é muito mais eficiente para objetos alocados na pilha do que para variáveis globais, e a alocação de registradores é praticamente impossível para objetos alocados na pilha heap, pois eles são acessados com ponteiros. As variáveis globais e algumas variáveis de pilha são impossíveis de alocar, já que usam aliases ou *nomes*

alternativos — existem várias maneiras de se referir ao endereço de uma variável, tornando inválida colocá-la em um registrador. (A maioria das variáveis de heap, na prática, recebe nomes alternativos eficientes para a tecnologia de compilador atual.)

Por exemplo, considere esta sequência de código, em que `&` retorna o endereço de uma variável e `*` desreferencia um ponteiro:

```
p = &a-- -- obtém endereço de a em p
a = ...-- atribui diretamente à variável a
*p = ...-- usa p para atribuir à variável a
...a...-- acessa a
```

A variável `a` não poderia ser alocada a um registrador pela atribuição a `*p` sem gerar código incorreto. O uso de um nome alternativo causa um grande problema, porque normalmente é difícil ou impossível decidir a quais objetos um ponteiro pode se referir. Um compilador precisa ser conservador; alguns compiladores não alocarão *quaisquer* variáveis locais de um procedimento em um registrador quando houver um ponteiro que possa se referir a *uma* das variáveis locais.

Como o arquiteto pode ajudar o projetista de compiladores

Hoje, a complexidade de um compilador não vem de traduzir declarações simples, como $A = B + C$. A maioria dos programas é *localmente simples* e as traduções simples funcionam bem. Em vez disso, a complexidade surge porque os programas são grandes e globalmente complexos em suas interações e porque a estrutura dos compiladores significa que, em cada etapa, são tomadas decisões sobre qual é a melhor sequência de código.

Os projetistas de compiladores normalmente estão trabalhando sob seu próprio corolário de um princípio básico na arquitetura: *torne os casos frequentes rápidos e os casos raros corretos*. Ou seja, se soubermos que casos são frequentes e quais deles são raros, e se a geração de código para ambos for direta, então a qualidade do código para o caso raro pode não ser muito importante, mas ele precisa estar correto!

Algumas propriedades de conjunto de instruções ajudam o projetista de compiladores. Essas propriedades não devem ser consideradas regras rígidas, mas orientações que facilitarão a escrita de um compilador que gere um código eficiente e correto.

- *Proporcionar regularidade.* Sempre que fizer sentido, os três componentes primários de um conjunto de instruções — as operações, os tipos de dados e os modos de endereçamento — devem ser *ortogonais*. Dizemos que dois aspectos de uma arquitetura são ortogonais se eles forem independentes. Por exemplo, as operações e os modos de endereçamento são ortogonais se, para cada operação à qual um modo de endereçamento puder ser aplicado, todos os modos de endereçamento forem aplicáveis. Essa regularidade ajuda a simplificar a geração de código e é particularmente importante quando a decisão sobre que código gerar é dividida em duas passagens no compilador. Um bom contraexemplo dessa propriedade é restringir os registradores que podem ser usados para certa classe de instruções. Os compiladores para arquiteturas de registrador de propósito especial normalmente ficam presos nesse dilema. A restrição pode fazer com que o compilador acabe com um monte de registradores disponíveis, mas nenhum do tipo certo!
- *Fornecer primitivas, não soluções.* Os recursos especiais que “correspondem” a uma construção de linguagem ou a *uma* função de kernel normalmente são inúteis. As tentativas de dar suporte a linguagens de alto nível podem funcionar apenas com

uma linguagem ou fazer mais ou menos o que é necessário para uma implementação correta e eficiente da linguagem. Um exemplo de como essas tentativas falharam é dado na [Seção A.10](#).

- *Simplificar os compromissos entre alternativas.* Uma das tarefas mais difíceis de um projetista de compilador é descobrir que sequência de instruções será a melhor para cada segmento de código que surge. Antigamente, as contagens de instruções ou o tamanho de código total podem ter sido boas métricas, mas — como vimos no Capítulo 1 — isso não é mais verdade. Com caches e pipelining, os compromissos se tornaram muito complexos. Qualquer coisa que o arquiteto puder fazer para ajudar o projetista de compilador a entender os custos das sequências alternativas de código ajudará a melhorar o código. Uma das instâncias mais difíceis dos compromissos complexos ocorre na arquitetura de registrador-memória, ao se decidir quantas vezes uma variável deve ser referenciada antes que seja menos oneroso carregá-la em um registrador. É difícil calcular esse limiar; na verdade, pode variar entre modelos da mesma arquitetura.
- *Fornecer instruções que associem as quantidades conhecidas em tempo de compilação como constantes.* Um projetista de compilador detesta a ideia de o processador interpretar, em tempo de execução, um valor que foi conhecido em tempo de compilação. Bons contraexemplos desse princípio incluem instruções que interpretam valores que foram fixados em tempo de compilação. Por exemplo, a chamada de procedimento do VAX (calls) interpreta dinamicamente uma máscara dizendo quais registradores salvar em uma chamada, mas a máscara é fixada em tempo de compilação ([Seção A.10](#)).

O suporte do compilador (ou a falta dele) para instruções de multimídia

Infelizmente, os projetistas das instruções SIMD ([Seção 4.3](#), no Cap. 4) basicamente ignoraram a subseção anterior. Essas instruções costumam ser soluções, não primitivas; elas têm poucos registradores; e os tipos de dados não correspondem às linguagens de programação existentes. Os arquitetos esperavam encontrar uma solução barata que ajudasse a alguns usuários, mas, na realidade, apenas algumas rotinas de biblioteca gráfica de baixo nível as utilizam.

As instruções SIMD são, na verdade, uma versão abreviada de um estilo de arquitetura elegante que possui sua própria tecnologia de compilador. Como explicamos na [Seção 4.2](#), as *arquitecturas vetoriais* operam em vetores de dados. Criados originalmente para códigos científicos, em geral os kernels de multimídia são vetorizáveis também, embora frequentemente com vetores menores. Como a [Seção 4.3](#) sugere, podemos pensar no MMX e SSE da Intel ou no PowerPC AltiVec simplesmente como computadores de vetor curto: o MMX, com vetores de oito elementos de 8 bits, quatro elementos de 16 bits ou dois elementos de 32 bits, e o AltiVec, com vetores com o dobro disso. Eles são implementados simplesmente como elementos estreitos adjacentes em grandes registradores.

Essas arquiteturas de microprocessador aumentaram o tamanho de registrador vetorial na arquitetura: a soma dos tamanhos dos elementos é limitada a 64 bits para o MMX e a 128 bits para o AltiVec. Quando a Intel decidiu expandir para vetores de 128 bits, ela acrescentou todo um novo conjunto de instruções, chamado Streaming SIMD Extension (SSE).

Uma grande vantagem dos computadores vetoriais é ocultar a latência do acesso à memória carregando muitos elementos de uma vez e, depois, sobrepondo a execução com transferência de dados. O objetivo dos modos de endereçamento vetoriais é coletar dados espalhados na memória, colocá-los em uma forma compacta para que possam ser operados eficientemente e, depois, colocar os resultados nos lugares a que pertencem.

Os computadores vetoriais tradicionais acrescentaram *endereçamento espaçado* e *endereçamento gather-scatter* (Seção 4.2) para aumentar o número de programas que podem ser vetorizados. O endereçamento espaçado salta um número fixo de palavras entre cada acesso, de modo que o endereçamento sequencial normalmente é chamado *endereçamento espaço único*. *Gather-scatter* encontra seus endereços em outro registrador vetorial: pense nele como endereçamento indireto de registrador para computadores vetoriais. Por outro lado, sob uma perspectiva vetorial, esses computadores SIMD de vetor curto aceitam apenas acessos com espaçamento unitário: os acessos à memória carregam ou armazenam todos os elementos ao mesmo tempo de um único grande espaço da memória. Como os dados para aplicações de multimídia geralmente são fluxos que começam e terminam na memória, os modos de endereçamento avançado e *gather-scatter* são essenciais para uma vetorização bem-sucedida (Seção 4.7).

Exemplo Como exemplo, compare um computador de vetor com o MMX para conversão de representação de cor de pixels de RGB (vermelho, verde, azul) para YUV (luminosidade, cromaticidade), com cada pixel representado por 3 bytes. A conversão envolve apenas três linhas de código C colocadas em um loop:

$$\begin{aligned} Y &= (9798 * R + 19235 * G + 3736 * B) / 32768; \\ U &= (-4784 * R - 9437 * G + 4221 * B) / 32768 + 128; \\ V &= (20218 * R - 16941 * G - 3277 * B) / 32768 + 128; \end{aligned}$$

Um computador vetorial de 64 bits pode calcular 8 pixels simultaneamente. Um computador vetorial para mídia com endereços espaçados usa:

- 3 cargas vetoriais (para obter RGB)
- 3 multiplicações vetoriais (para converter R)
- 6 multiplicações adições vetoriais (para converter G e B)
- 3 deslocamentos vetoriais (para dividir por 32.768)
- 2 adições vetoriais (para somar 128)
- 3 armazenamentos vetoriais (para armazenar YUV)

O total é de 20 instruções para realizar as 20 operações no código C anterior, para converter 8 pixels (Kozyrakis, 2000). (Como um vetor pode ter 32 elementos de 64 bits, esse código, na verdade, converte até $32 \times 8 = 256$ pixels.) Por sua vez, o site da Intel mostra que uma rotina de biblioteca para realizar o mesmo cálculo em 8 pixels leva 116 instruções MMX mais seis instruções 80x86 (Intel, 2001). Esse aumento de seis vezes nas instruções é devido ao grande número de instruções para carregar e desempacotar pixels RGB e para empacotar e armazenar pixels YUV, já que não existem acessos espaçados à memória.

Ter vetores curtos e limitados à arquitetura com poucos registradores e modos de endereçamento de memória simples torna mais difícil usar tecnologia de compilador de vetorização. Assim, essas instruções SIMD têm maior probabilidade de serem encontradas em bibliotecas codificadas à mão do que em código compilado.

Resumo: o papel dos compiladores

Esta seção leva a várias recomendações: primeiro, esperamos que uma nova arquitetura de conjunto de instruções tenha pelo menos 16 registradores de propósito geral — além de registradores separados para números de ponto flutuante — para simplificar a alocação de registradores usando coloração de grafo. O conselho sobre a ortogonalidade sugere que todos os modos de endereçamento aceitos se aplicam a todas as instruções que transferem dados. Finalmente, os três últimos conselhos — fornecer primitivas em vez de soluções, simplificar compromissos entre alternativas e não associar constantes

em tempo de execução — sugerem que é melhor errar pela simplicidade. Em outras palavras, entenda que menos é melhor no projeto de um conjunto de instruções. Infelizmente, as extensões SIMD são mais um exemplo de bom marketing do que um grande sucesso do coprojetado de hardware-software.

A.9 JUNTANDO TUDO: A ARQUITETURA MIPS

Nesta seção, descreveremos uma arquitetura de carregamento-armazenamento (load-store) simples de 64 bits, chamada MIPS. A arquitetura de conjunto de instruções das relativas ao MIPS e ao RISC foi baseada em observações semelhantes às discutidas nas últimas seções (na Seção L.3 discutiremos como e por que essas arquiteturas se tornaram populares). Revendo nossas expectativas de cada seção, para aplicações de desktop:

- **Seção A.2.** Usar registradores de propósito geral com uma arquitetura de carregamento-armazenamento.
- **Seção A.3.** Aceitar esses modos de endereçamento: deslocamento (com tamanho de deslocamento de endereço de 12-16 bits), imediato (tamanho de 8-16 bits) e indireto de registrador.
- **Seção A.4.** Aceitar esses tamanhos e tipos de dados: inteiros de 8, 16, 32 e 64 bits e números de ponto flutuante IEEE 754 de 64 bits.
- **Seção A.5.** Aceitar essas instruções simples, já que elas dominarão o número de instruções executadas: carregamento, armazenamento, adição, subtração, mover registrador-registrador e deslocamento.
- **Seção A.6.** Comparação de igualdade, comparação de desigualdade, comparação de menor que, desvio (com um endereço relativo ao PC de pelo menos objeto), salto, chamada e retorno de procedimento.
- **Seção A.7.** Usar codificação de instrução fixa, se estiver interessado em desempenho, e codificação de instrução variável, se estiver interessado em tamanho de código.
- **Seção A.8.** Fornecer pelo menos 16 registradores de propósito geral, certificar que todos os modos de endereçamento se aplicam a todas as instruções de transferência de dados e objetivar um conjunto mínimo de instruções. Esta seção não aborda programas de ponto flutuante, mas eles normalmente usam registradores de ponto flutuante separados. A justificativa é aumentar o número de registradores sem causar problemas no formato de instrução ou na velocidade do banco de registradores (register file) de propósito geral. Esse compromisso, no entanto, não é ortogonal.

Introduzimos o MIPS mostrando como ele segue essas recomendações. Como a maioria dos computadores atuais, o MIPS enfatiza:

- Um conjunto de instruções de carregamento-armazenamento simples.
- Um projeto que visa à eficiência do *pipelining* (discutido no Apêndice C), incluindo uma codificação fixa de conjunto de instruções.
- Eficiência como uma meta do compilador.

O MIPS fornece um bom modelo arquitetônico para estudo, não só devido à popularidade desse tipo de processador, mas porque essa é uma arquitetura fácil de entender. Usamos essa arquitetura também no Apêndice C e no Capítulo 3, e ela forma a base para diversos exercícios e projetos de programação.

Desde o primeiro processador MIPS em 1985, tem havido muitas versões do MIPS (Apêndice K). Usaremos um subconjunto do que hoje é chamado MIPS64, que normalmente será abreviado para MIPS, mas o conjunto de instruções completo é encontrado no Apêndice K.

Registradores para MIPS

O MIPS64 possui 32 registradores de propósito geral (GPRs) de 64 bits, denominados R0, R1, ..., R31. Às vezes, os GPRs são chamados *registradores de inteiros*. Além disso, existe um conjunto de 32 registradores de ponto flutuante (FPRs), denominados F0, F1, ..., F31, que podem armazenar 32 valores de precisão simples (32 bits) ou valores de precisão dupla (64 bits). (Quando armazena um número de precisão simples, a outra metade do FPR fica ociosa.) Tanto as operações de ponto flutuante de precisão simples quanto as de precisão dupla são disponibilizadas. O MIPS também inclui instruções que operam em dois operandos de precisão simples em um único registrador de ponto flutuante de 64 bits.

O valor de R0 é sempre 0. Mais adiante, veremos como podemos usar esse registrador para sintetizar uma variedade de operações úteis a partir de um conjunto de instruções simples.

Alguns registradores especiais podem ser transformados de e para os registradores de propósito geral. Um exemplo disso é o registrador de status de ponto flutuante, usado para guardar informações sobre os resultados das operações de ponto flutuante. Também existem instruções para mover entre um FPR e um GPR.

Tipos de dados para MIPS

Os tipos de dados são bytes de 8 bits, meias palavras de 16 bits, palavras de 32 bits, palavras duplas de 64 bits para dados de inteiro, precisão simples de 32 bits e precisão dupla de 64 bits para ponto flutuante. As meias palavras foram acrescentadas porque são encontradas em linguagens como C e são populares em alguns programas, como os sistemas operacionais, preocupados com o tamanho das estruturas de dados. Elas também se tornarão mais comuns se o Unicode se tornar bastante usado. Os operandos de ponto flutuante de precisão simples foram acrescentados por questões semelhantes. (Lembre-se da advertência inicial de que você deve medir muitos outros programas antes de projetar um conjunto de instruções.)

Os operandos do MIPS64 funcionam em inteiros de 64 bits e ponto flutuante de 32 ou 64 bits. Bytes, meias palavras e palavras são carregados nos registradores de propósito geral com zeros ou o bit de sinal replicado para preencher os 64 bits dos GPRs. Uma vez carregados, eles são usados em operações de inteiros de 64 bits.

Modos de endereçamento para transferências de dados MIPS

Os únicos modos de endereçamento de dados são o modo imediato e o de deslocamento, ambos com campos de 16 bits. O modo indireto de registrador é conseguido simplesmente colocando 0 no campo de deslocamento de 16 bits, e o endereçamento absoluto com um campo de 16 bits é realizado usando o registrador 0 como o registrador-base. Usar zero nos dá quatro modos efetivos, embora apenas dois sejam aceitos na arquitetura.

A memória MIPS é endereçável por byte com um endereço de 64 bits. Ela tem um bit de modo que permite ao software selecionar Big Endian ou Little Endian. Por se tratar de uma arquitetura de carregamento-armazenamento (load-store), todas as referências entre a memória e os GPRs ou FPRs são através de carregamentos ou armazenamentos. Aceitando os tipos de dados mencionados, os acessos à memória envolvendo GPRs podem ser para um byte, meia palavra ou palavra dupla. Os FPRs podem ser carregados e armazenados com números de precisão simples ou precisão dupla. Todos os acessos à memória precisam ser alinhados.

Formato de instrução do MIPS

Como o MIPS possui apenas dois modos de endereçamento, eles podem ser codificados no opcode. Seguindo o conselho sobre tornar fácil o pipeline e a decodificação do processador, todas as instruções são de 32 bits com um opcode primário de 6 bits. A [Figura A.22](#) mostra o leiaute da instrução. Esses formatos são simples, embora fornecendo campos de 16 bits para endereçamento de deslocamento, constantes imediatas ou endereços de desvio relativos ao PC.

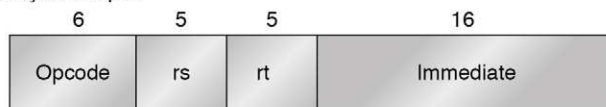
O Apêndice K mostra uma variante do MIPS — chamada MIPS16 — que possui instruções de 16 bits e 32 bits para melhorar a densidade de código para aplicações embarcadas. Ficaremos com o formato de 32 bits tradicional neste livro.

Operações MIPS

O MIPS aceita a lista das operações simples recomendada a seguir, além de algumas outras. Existem quatro grandes classes de instruções: carregamentos e armazenamentos, operações da ULA, desvios e saltos, e operações de ponto flutuante.

Qualquer um dos registradores de propósito geral ou de ponto flutuante pode ser carregado ou armazenado, exceto que o carregamento de R0 não tem qualquer efeito. A [Figura A.23](#) dá exemplos das instruções carregamento e armazenamento. Os números de ponto flutuante de precisão simples ocupam metade de um registrador de ponto flutuante. As conversões entre precisão simples e dupla precisam ser feitas explicitamente. O formato de ponto flutuante é o IEEE 754 (Apêndice J). Uma lista de todas as instruções MIPS em nosso subconjunto aparece na [Figura A.26](#) (página A-35).

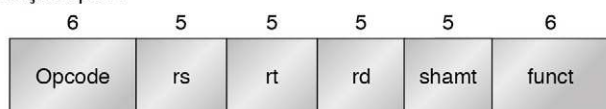
Instrução do tipo I



Encodes: Carregamentos e armazenamentos de bytes, meias palavras, palavras duplas. Todos os imediatos ($RT \leftarrow rs \text{ op } \text{immediate}$)

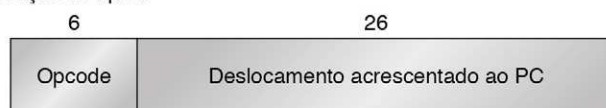
Instruções de desvio condicional (rs é registrador, rd não usado)
Registrador de salto, registrador salto e link
($rd=0$, $rs=\text{destino}$, $\text{imediato}=0$)

Instrução tipo R



Operandos de ALU registrador-registrador: $rd \leftarrow rs \text{ funct } rt$
Função codifica a operação de caminho de dados: Add, Sub, . . .
Registradores e movimentos especiais de leitura/escrita

Instrução de tipo J



Jump and jump and link
Trap and return from exception

FIGURA A.22 Leiaute de instrução para MIPS.

Todas as instruções são codificadas em um de três tipos, com campos comuns no mesmo local em cada formato.

Instrução de exemplo	Nome da instrução	Significado
LD R1, 30(R2)	Load double word	$\text{Regs}[\text{R1}] \leftarrow_{64} \text{Mem}[\text{30} + \text{Regs}[\text{R2}]]$
LD R1, 1000(R0)	Load double word	$\text{Regs}[\text{R1}] \leftarrow_{64} \text{Mem}[\text{1000} + 0]$
LW R1, 60(R2)	Load word	$\text{Regs}[\text{R1}] \leftarrow_{64} (\text{Mem}[\text{60} + \text{Regs}[\text{R2}]]_0)^{32} \text{ ## Mem}[\text{60} + \text{Regs}[\text{R2}]]$
LB R1, 40(R3)	Load byte	$\text{Regs}[\text{R1}] \leftarrow_{64} (\text{Mem}[\text{40} + \text{Regs}[\text{R3}]]_0)^{56} \text{ ## Mem}[\text{40} + \text{Regs}[\text{R3}]]$
LBU R1, 40(R3)	Load byte unsigned	$\text{Regs}[\text{R1}] \leftarrow_{64} 0^{56} \text{ ## Mem}[\text{40} + \text{Regs}[\text{R3}]]$
LH R1, 40(R3)	Load half word	$\text{Regs}[\text{R1}] \leftarrow_{64} (\text{Mem}[\text{40} + \text{Regs}[\text{R3}]]_0)^{48} \text{ ## Mem}[\text{40} + \text{Regs}[\text{R3}]] \text{ ## Mem}[\text{41} + \text{Regs}[\text{R3}]]$
L.S F0, 50(R3)	Load FP single	$\text{Regs}[\text{F0}] \leftarrow_{64} \text{Mem}[\text{50} + \text{Regs}[\text{R3}]] \text{ ## } 0^{32}$
L.D F0, 50(R2)	Load FP double	$\text{Regs}[\text{F0}] \leftarrow_{64} \text{Mem}[\text{50} + \text{Regs}[\text{R2}]]$
SD R3, 500(R4)	Store double word	$\text{Mem}[\text{500} + \text{Regs}[\text{R4}]] \leftarrow_{64} \text{Regs}[\text{R3}]$
SW R3, 500(R4)	Store word	$\text{Mem}[\text{500} + \text{Regs}[\text{R4}]] \leftarrow_{32} \text{Regs}[\text{R3}]_{32..63}$
S.S F0, 40(R3)	Store FP single	$\text{Mem}[\text{40} + \text{Regs}[\text{R3}]] \leftarrow_{32} \text{Regs}[\text{F0}]_{0..31}$
S.D F0, 40(R3)	Store FP double	$\text{Mem}[\text{40} + \text{Regs}[\text{R3}]] \leftarrow_{64} \text{Regs}[\text{F0}]$
SH R3, 502(R2)	Store half	$\text{Mem}[\text{502} + \text{Regs}[\text{R2}]] \leftarrow_{16} \text{Regs}[\text{R3}]_{48..63}$
SB R2, 41(R3)	Store byte	$\text{Mem}[\text{41} + \text{Regs}[\text{R3}]] \leftarrow_8 \text{Regs}[\text{R2}]_{56..63}$

FIGURA A.23 As instruções carregamento e armazenamento (load e store) no MIPS.

Todas usam um único modo de endereçamento e exigem que o valor de memória esteja alinhado. Obviamente, tanto os carregamentos quanto os armazenamentos estão disponíveis para todos os tipos de dados mostrados.

Para entender essas figuras, precisamos introduzir algumas extensões adicionais em nossa linguagem de descrição C, usada inicialmente na página A-8:

- Um subscrito é anexado ao símbolo $\leftarrow$ sempre que o tamanho do dado que está sendo transferido pode não ser claro. Assim, $\leftarrow_n$ significa uma quantidade de n bits. Usamos $x, y \leftarrow z$ para indicar que z deve ser transferido para x e y .
- Um subscrito é usado para indicar a seleção de um bit de um campo. Os bits são rotulados a partir do bit mais significativo começando em 0. O subscrito pode ser um único dígito (p. ex., $\text{Regs}[\text{R4}]_0$ produz o bit de sinal R4) ou uma subfaixa (p. ex., $\text{Regs}[\text{R3}]_{56..63}$ produz o byte menos significativo de R3).
- A variável *Mem*, usada como um array que representa a memória principal, é indexada por um endereço de byte e pode transferir qualquer número de bytes.
- Um sobrescrito é usado para reproduzir um campo (p. ex., 0^{48} produz um campo de zeros de 48 bits de extensão).
- O símbolo **##** é usado para concatenar dois campos e pode aparecer em qualquer lado de uma transferência de dados.

Como exemplo, considerando que R8 e R10 são registradores de 64 bits:

$$\text{Regs}[\text{R10}]_{32..63} \leftarrow_{32} (\text{Mem}[\text{Regs}[\text{R8}]]_0)^{24} \text{ ## Mem}[\text{Regs}[\text{R8}]]$$

significa que o byte no local de memória endereçado pelo conteúdo do registrador R8 é estendido com o sinal para formar uma quantidade de 32 bits, que é armazenada na metade inferior do registrador R10 (a metade superior de R10 fica inalterada).

Todas as instruções da ULA são instruções registrador-registrador. A [Figura A.24](#) fornece alguns exemplos das instruções aritméticas/lógicas. As operações incluem aritmética simples e operações lógicas: adição, subtração, AND, OR, XOR e deslocamentos. As formas imediatas de todas essas instruções são fornecidas usando um imediato de 16 bits es-

Instrução de exemplo	Nome da instrução	Significado
DADDU R1,R2,R3	Add unsigned	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R2}] + \text{Regs}[\text{R3}]$
DADDIU R1,R2,#3	Add immediate unsigned	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R2}] + 3$
LUI R1,#42	Load upper immediate	$\text{Regs}[\text{R1}] \leftarrow 0^{32} \#42 \#0^{16}$
DSLL R1,R2,#5	Shift left logical	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R2}] \ll 5$
SLT R1,R2,R3	Set less than	if ($\text{Regs}[\text{R2}] < \text{Regs}[\text{R3}]$) $\text{Regs}[\text{R1}] \leftarrow 1$ else $\text{Regs}[\text{R1}] \leftarrow 0$

FIGURA A.24 Exemplos das instruções aritméticas/lógicas no MIPS, com e sem imediatos.

tendido com o sinal. A operação LUI (load upper immediate — carregar imediato superior) carrega os bits 32 a 47 de um registrador, enquanto define o restante do registrador em 0. LUI permite que uma constante de 32 bits seja construída em duas instruções ou uma transferência de dados usando qualquer endereço constante de 32 bits em uma instrução extra.

Como mencionamos, R0 é usado para sintetizar operações populares. Carregar uma constante é simplesmente um add imediato onde o operando de origem é R0, e um move registrador-registrador é simplesmente um add onde uma das origens é R0 (algumas vezes usamos o mnemônico LI, significando load immediate [carregar imediato], para representar o primeiro e o mnemônico MOV para o último).

Instruções de fluxo de controle do MIPS

O MIPS fornece instruções de comparação, que comparam dois registradores para ver se o primeiro é menor que o segundo. Se a condição for verdadeira, essas instruções colocam um 1 no registrador de destino (para representar verdadeiro); caso contrário, elas colocam o valor 0. Como essas operações “definem” um registrador, elas são chamadas set-equal, set-not-equal, set-less-than, e assim por diante. Também há formas imediatas dessas comparações.

O controle é manipulado através de um conjunto de saltos e um conjunto de desvios. A [Figura A.25](#) fornece algumas instruções típicas de desvio e de salto. As quatro instruções de salto são diferenciadas pelas duas maneiras de especificar o endereço de destino e pelo

Instrução de exemplo	Nome da instrução	Significado
J name	Jump	$\text{PC}_{36..63} \leftarrow \text{name}$
JAL name	Jump and link	$\text{Regs}[\text{R31}] \leftarrow \text{PC} + 8$; $\text{PC}_{36..63} \leftarrow \text{name}$; $((\text{PC} + 4) - 2^{27}) \leq \text{name} < ((\text{PC} + 4) + 2^{27})$
JALR R2	Jump and link register	$\text{Regs}[\text{R31}] \leftarrow \text{PC} + 8$; $\text{PC} \leftarrow \text{Regs}[\text{R2}]$
JR R3	Jump register	$\text{PC} \leftarrow \text{Regs}[\text{R3}]$
BEQZ R4,name	Branch equal zero	If ($\text{Regs}[\text{R4}] == 0$) $\text{PC} \leftarrow \text{name}$; $((\text{PC} + 4) - 2^{17}) \leq \text{name} < ((\text{PC} + 4) + 2^{17})$
BNE R3,R4,name	Branch not equal zero	if ($\text{Regs}[\text{R3}] \neq \text{Regs}[\text{R4}]$) $\text{PC} \leftarrow \text{name}$; $((\text{PC} + 4) - 2^{17}) \leq \text{name} < ((\text{PC} + 4) + 2^{17})$
MOVZ R1,R2,R3	Conditional move if zero	if ($\text{Regs}[\text{R3}] == 0$) $\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R2}]$

FIGURA A.25 Instruções de fluxo de controle típicas do MIPS.

Todas as instruções de controle, exceto saltos para um endereço em um registrador, são relativas ao PC. Repare que as distâncias de desvio são maiores do que o campo de endereço sugeriria; já que as instruções MIPS são todas com 32 bits, o endereço de desvio de byte é multiplicado por 4 para obter uma distância maior.

fato de um link ser ou não ser feito. Dois saltos usam um offset de 26 bits, deslocado 2 bits e, depois, substituem os 28 bits inferiores do contador de programa (da instrução sequencialmente após o salto) para determinar o endereço de destino. As outras duas instruções de salto especificam um registrador que contém o endereço de destino. Existem dois tipos de saltos: salto simples e salto e link (usado para chamadas de procedimento). Este último coloca o endereço de retorno — o endereço da próxima instrução em sequência — em R31.

Todos os desvios são condicionais. A condição de desvio é especificada pela instrução, que deve testar se o registrador origem é zero ou não zero; o registrador pode conter um valor ou o resultado de uma comparação. Também existem instruções de desvio condicional para testar a igualdade entre dois registradores e se um registrador é negativo. O endereço de destino do desvio é especificado com um deslocamento sinalizado com 16 bits, que é deslocado dois bits à esquerda e, depois, adicionado ao contador de programa (PC), que aponta para a próxima instrução em sequência. Existe também um desvio para testar o registrador de status de ponto flutuante quanto aos desvios condicionais de ponto flutuante, descritos mais adiante.

O Apêndice C e o Capítulo 3 mostram que os desvios condicionais são um grande problema para execução em um pipeline; portanto, muitas arquiteturas acrescentaram instruções para converter um desvio simples em uma instrução aritmética condicional. O MIPS incluiu a movimentação condicional em zero ou não zero. O valor do registrador de destino é deixado inalterado ou é substituído por uma cópia de um dos registradores de origem, dependendo do fato de o valor do outro registrador de origem ser zero ou não.

Operações de ponto flutuante do MIPS

As instruções de ponto flutuante manipulam os registradores de ponto flutuante e indicam se a operação a ser realizada é de precisão simples ou dupla. As operações MOV. S e MOV. D copiam um registrador de ponto flutuante de precisão simples (MOV. S) ou de precisão dupla (MOV. D) para outro registrador do mesmo tipo. As operações MFC1, MTC1, DMFC1, DMTC1 movem dados entre um registrador de ponto flutuante simples ou duplo e um registrador inteiro. As conversões de inteiro para ponto flutuante também são fornecidas e vice-versa.

As operações de ponto flutuante são adição, subtração, multiplicação e divisão; um sufixo D é usado para precisão dupla, e um sufixo S é usado para precisão simples (p. ex., ADD. D, ADD. S, SUB. D, SUB. S, MUL. D, MUL. S, DIV. D, DIV. S). As comparações de ponto flutuante definem um bit no registrador especial de status de ponto flutuante que pode ser testado com um par de desvios: BC1T e BC1F, desvio de ponto flutuante verdadeiro e desvio de ponto flutuante falso.

Para conseguir maior desempenho para rotinas gráficas, o MIPS64 possui instruções que realizam duas operações de ponto flutuante de 32 bits em cada metade do registrador de ponto flutuante de 64 bits. Essas operações *simplesmente emparelhadas* incluem ADD. PS, SUB. PS, MUL. PS e DIV. PS (elas são carregadas e armazenadas usando carregamentos e armazenamentos de precisão dupla).

Acenando em direção à importância das aplicações de multimídia, o MIPS64 também inclui instruções multiplicação-adição de inteiro e ponto flutuante: MADD, MADD. S, MADD. D e MADD. PS. Os registradores são todos da mesma largura nessas operações combinadas. A [Figura A.26](#) contém uma lista de um subconjunto das operações MIPS64 e seu significado.

Tipo de instrução/opcode	Significado da instrução
<i>Transferências de dados</i>	<i>Move dados entre registradores e memória ou entre os registradores de inteiro e FP ou especial; apenas o modo de endereço de memória é deslocamento de 16 bits + conteúdo de um GPR</i>
LB, LBU, SB	Load byte, load byte não sinalizado, store byte (registradores de inteiro to/from)
LH, LHU, SH	Load half word, load half word não sinalizado, store half word (registradores de inteiro to/from)
LW, LWU, SW	Load word, load word não sinalizado, store word (registradores de inteiro to/from)
LD, SD	Load double word, store double word (registradores de inteiro to/from)
L.S, L.D, S.S, S.D	Load SP float, load DP float, store SP float, store DP float
MFC0, MTC0	Copiar de/para GPR para/de um registrador especial
MOV.S, MOV.D	Copiar um registrador FP SP ou DP para outro registrador FP
MFC1, MTC1	Copiar 32 bits para/de registradores FP de/para registradores de inteiro
<i>Aritmética/lógica</i>	<i>Operações em inteiros ou dados lógicos em GPRs; interceptação de aritmética com sinal no overflow</i>
DADD, DADDI, DADDU, DADDIU	Add, add immediate (todos os imediatos são de 16 bits); sinalizado e não sinalizado
DSUB, DSUBU	Subtração; sinalizado e não sinalizado
DMUL, DMULU, DDIV, DDIVU, MADD	Multiplicação e divisão, sinalizado e não sinalizado; multiplicação-adição; todas as operações recebem e produzem valores de 64 bits
AND, ANDI	And, and immediate
OR, ORI, XOR, XORI	Or, or immediate, exclusive or, exclusive or immediate
LUI	Load upper immediate; carrega os bits 32 a 47 do registrador com immediate, depois estende o sinal
DSLL, DSRL, DSRA, DSLLV, DSRLV, DSRAV	Shifts: forma immediate (DS__) e variável (DS__V); shifts são shift left logical, right logical, right arithmetic
SLT, SLTI, SLTU, SLTIU	Set less than, set less than immediate; sinalizado e não sinalizado
<i>Controle</i>	<i>Saltos e desvios condicionais; relativo ao PC ou através do registrador</i>
BEQZ, BNEZ	GPRs de desvio igual/não igual a zero; offset de 16 bits de PC + 4
BEQ, BNE	GPR de desvio igual/não igual; offset de 16 bits de PC + 4
BC1T, BC1F	Bit de comparação de teste no registrador de estado FP e desvio; offset de 16 bits de PC + 4
MOVN, MOVZ	Copia GPR para outro GPR se o terceiro GPR for negativo, zero
J, JR	Saltos: offset de 26 bits de PC + 4 (J) ou destino no registrador (JR)
JAL, JALR	Salto e link: salva PC + 4 em R31, destino é relativo ao PC (JAL) ou um registrador (JALR)
TRAP	Transfere para o sistema operacional em um endereço vetorizado
ERET	Retorna para o código do usuário de uma exceção; recupera o modo de usuário
<i>Ponto flutuante</i>	<i>Operações de FP nos formatos DP e SP</i>
ADD.D, ADD.S, ADD.PS	Soma números DP, SP e pares de números SP
SUB.D, SUB.S, SUB.PS	Subtrai números DP, SP e pares de números SP
MUL.D, MUL.S, MUL.PS	Multiplica ponto flutuante DP, SP e pares de números SP
MADD.D, MADD.S, MADD.PS	Multiplica-soma números DP, SP e pares de números SP
DIV.D, DIV.S, DIV.PS	Divide ponto flutuante DP, SP e pares de números SP
CVT.___	Converte instruções: CVT.x.y converte do tipo x para o tipo y, onde x e y são L (inteiro de 64 bits), W (inteiro de 32 bits), D (DP) ou S (SP). Ambos os operandos são FPRs.
C.___.D, C.___.S	Comparações DP e SP: “___” = LT,GT,LE,GE,EQ,NE; define o bit no registrador de estado FP

FIGURA A.26 Subconjunto das instruções no MIPS64.

A Figura A.22 lista os formatos dessas instruções. SP = precisão única; DP = precisão dupla.

Uso do conjunto de instruções MIPS

Para dar uma ideia de quais instruções são comuns, a [Figura A.27](#) mostra a frequência das instruções e classes de instrução para cinco programas SPECint2000, e a [Figura A.28](#) mostra os mesmos dados para cinco programas SPECfp2000.

A.10 FALÁCIAS E ARMADILHAS

Os arquitetos repetidamente tropeçam em crenças comuns, mas errôneas. Nesta seção veremos algumas delas.

Instrução	gap	gcc	gzip	mcf	perlbnk	Média de inteiro
load	26,5%	25,1%	20,1%	30,3%	28,7%	26%
store	10,3%	13,2%	5,1%	4,3%	16,2%	10%
add	21,1%	19,0%	26,9%	10,1%	16,7%	19%
sub	1,7%	2,2%	5,1%	3,7%	2,5%	3%
mul	1,4%	0,1%				0%
compare	2,8%	6,1%	6,6%	6,3%	3,8%	5%
load imm	4,8%	2,5%	1,5%	0,1%	1,7%	2%
cond desvio	9,3%	12,1%	11,0%	17,5%	10,9%	12%
cond move	0,4%	0,6%	1,1%	0,1%	1,9%	1%
salto	0,8%	0,7%	0,8%	0,7%	1,7%	1%
call	1,6%	0,6%	0,4%	3,2%	1,1%	1%
return	1,6%	0,6%	0,4%	3,2%	1,1%	1%
shift	3,8%	1,1%	2,1%	1,1%	0,5%	2%
AND	4,3%	4,6%	9,4%	0,2%	1,2%	4%
OR	7,9%	8,5%	4,8%	17,6%	8,7%	9%
XOR	1,8%	2,1%	4,4%	1,5%	2,8%	3%
other logical	0,1%	0,4%	0,1%	0,1%	0,3%	0%
load FP						0%
store FP						0%
add FP						0%
sub FP						0%
mul FP						0%
div FP						0%
mov reg-reg FP						0%
compare FP						0%
cond mov FP						0%
other FP						0%

FIGURA A.27 Mix de instrução dinâmica MIPS para cinco programas SPECint2000.

Observe que as instruções move registrador-registrador de inteiro são incluídas na instrução OR. Entradas em branco possuem o valor 0,0%.

Instrução	applu	art	equake	lucas	swim	Média de FP
load	13,8%	18,1%	22,3%	10,6%	9,1%	15%
store	2,9%		0,8%	3,4%	1,3%	2%
add	30,4%	30,1%	17,4%	11,1%	24,4%	23%
sub	2,5%		0,1%	2,1%	3,8%	2%
mul	2,3%			1,2%		1%
compare		7,4%	2,1%			2%
load imm	13,7%		1,0%	1,8%	9,4%	5%
cond desvio	2,5%	11,5%	2,9%	0,6%	1,3%	4%
cond move		0,3%	0,1%			0%
salto			0,1%			0%
call			0,7%			0%
return			0,7%			0%
shift	0,7%		0,2%	1,9%		1%
AND			0,2%	1,8%		0%
OR	0,8%	1,1%	2,3%	1,0%	7,2%	2%
XOR		3,2%	0,1%			1%
other logical			0,1%			0%
carregamento FP	11,4%	12,0%	19,7%	16,2%	16,8%	15%
armazenamento FP	4,2%	4,5%	2,7%	18,2%	5,0%	7%
add FP	2,3%	4,5%	9,8%	8,2%	9,0%	7%
sub FP	2,9%		1,3%	7,6%	4,7%	3%
mul FP	8,6%	4,1%	12,9%	9,4%	6,9%	8%
div FP	0,3%	0,6%	0,5%		0,3%	0%
mov reg-reg FP	0,7%	0,9%	1,2%	1,8%	0,9%	1%
compare FP		0,9%	0,6%	0,8%		0%
cond mov FP		0,6%		0,8%		0%
other FP				1,6%		0%

FIGURA A.28 Mix de instrução dinâmica MIPS para cinco programas do SPECfp2000.

Observe que as instruções move registrador-registrador de inteiro são incluídas na instrução or. Entradas em branco possuem o valor 0,0%.

Armadilha. *Projetar um recurso de conjunto de instruções de “alto nível” especificamente orientado para dar suporte a uma estrutura de linguagem de alto nível.*

As tentativas de incorporar recursos de linguagem de alto nível no conjunto de instruções levaram os arquitetos a disponibilizar instruções poderosas com grande flexibilidade. Entretanto, muitas vezes, essas instruções fazem mais do que é necessário no caso frequente ou não atendem exatamente às necessidades de algumas linguagens. Muitos desses esforços visavam à eliminação do que, na década de 1970, era chamado *semantic gap*. Embora a

ideia seja suplementar o conjunto de instruções com adições que trazem o hardware até o nível da linguagem, as adições podem gerar o que Wulf (1981) chamou de *semantic clash*:

[...] fornecendo conteúdo semântico excessivo à instrução, o projetista de computador possibilitava o uso da instrução apenas em contextos limitados [pág. 43].

Muitas vezes, as instruções são simplesmente arrasadoras — elas são gerais demais para o caso mais frequente, resultando em trabalho desnecessário e uma instrução mais lenta. Novamente, a **CALLS** do VAX é um bom exemplo. A **CALLS** usa uma estratégia de gravação pelo procedimento chamado (os registradores a serem salvos são especificados pelo procedimento chamado), *mas* o salvamento é feito pela instrução **call** no de quem chama. A instrução **CALLS** começa com os argumentos colocados na pilha e, depois, realiza as seguintes etapas:

1. Alinhar a pilha, se necessário.
2. Colocar a contagem de argumento na pilha.
3. Salvar os registradores indicados pela máscara de chamada de procedimento na pilha (como mencionado na [Seção A.8](#)). A máscara é mantida no código do procedimento chamado — isso permite que o procedimento chamado especifique os registradores a serem salvos por quem chamou, mesmo com uma compilação separada.
4. Colocar o endereço de retorno na pilha e, depois, enviar o topo e a base dos ponteiros da pilha (para o registro de ativação).
5. Limpar os códigos de condição, o que define a habilitação de trap para um status conhecido.
6. Colocar uma palavra para informação de status e uma palavra zero na pilha.
7. Atualizar os dois ponteiros de pilha.
8. Desviar para a primeira instrução do procedimento.

A grande maioria das chamadas nos programas reais não exige essa quantidade de overhead. A maioria dos procedimentos conhece suas contagens de argumento, e uma convenção de encadeamento muito mais rápida pode ser estabelecida usando registradores para passar argumentos em vez da pilha na memória. Além disso, a instrução **CALLS** força dois registradores a serem usados para o encadeamento, enquanto muitas linguagens exigem apenas um registrador de encadeamento. Muitas tentativas de aceitar gerenciamento de chamada de procedimento e de ativação de pilha falharam, quer porque não atendiam às necessidades da linguagem, quer porque eram muito gerais e, portanto, muito onerosas para usar.

Os projetistas do VAX forneceram uma instrução simples, **JSB**, que é muito mais rápida, já que coloca o PC de retorno na pilha e salta para o procedimento. Entretanto, a maioria dos compiladores VAX usa as instruções **CALLS** mais onerosas. As instruções de chamada foram incluídas na arquitetura para padronizar a convenção de encadeamento de procedimento. Outros computadores padronizaram sua convenção de chamada por acordo entre os projetistas de compilador e sem exigir o overhead de uma instrução de chamada de procedimento complexa e muito geral.

Falácia. Existe um programa típico.

Muitas pessoas gostam de acreditar que existe um único programa “típico” que poderia ser usado para projetar um conjunto de instruções ótimo. Por exemplo, veja os benchmarks sintéticos discutidos no Capítulo 1. Os dados neste Apêndice mostram claramente que os programas podem variar de modo significativo na maneira como usam um conjunto de instruções. Por exemplo, a [Figura A.29](#) mostra o mix dos tamanhos da transferência de dados para quatro programas do SPEC2000: seria difícil dizer o que é típico nesses programas. As variações são ainda maiores em um conjunto de instruções que aceita uma classe de aplicações, como instruções de decimal, que não são usadas por outras aplicações.

Armadilha. Inovar na arquitetura de conjunto de instruções para reduzir o tamanho do código sem levar em conta o compilador.

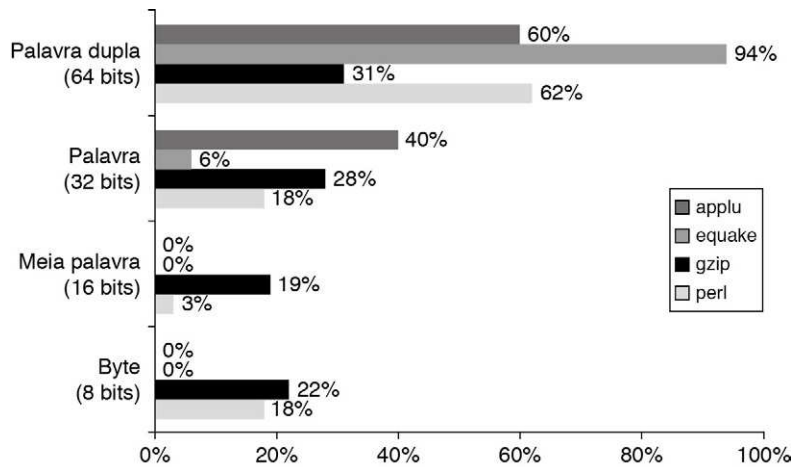


FIGURA A.29 Tamanho da referência de dados de quatro programas do SPEC2000. Embora seja possível calcular um tamanho médio, seria difícil afirmar que a média é típica dos programas.

A [Figura A.30](#) mostra os tamanhos de código relativos de quatro compiladores para o conjunto de instruções MIPS. Enquanto os arquitetos se esforçam para reduzir o tamanho de código em 30-40%, diferentes estratégias de compilador podem mudar o tamanho de código por fatores muito maiores. Assim como no que se refere às técnicas de otimização de desempenho, o arquiteto deve começar com o código mais compacto que os compiladores podem produzir antes de propor inovações de hardware para ganhar espaço.

Falácia. *Uma arquitetura com falhas não pode ser bem-sucedida.*

O 80x86 oferece um exemplo drástico: a arquitetura de conjunto de instruções é uma que só poderia ser adorada por seus criadores (Apêndice K). Sucessivas gerações de engenheiros da Intel tentaram corrigir as decisões arquitetônicas impopulares tomadas no projeto do 80x86. Por exemplo, a arquitetura aceita a segmentação, enquanto todas as outras escolheram a paginação; ela usa acumuladores estendidos para dados de inteiro, enquanto outros processadores usam registradores de propósito geral; ela usa uma pilha para dados de ponto flutuante, quando todo mundo abandonou as pilhas de execução há muito tempo.

Compilador	Apogee Software Versão 4.1	Green Hills Multi2000 Versão 2.0	Algorithmics SDE4.0B	IDT/c 7.2.1
Arquitetura	MIPS IV	MIPS IV	MIPS 32	MIPS 32
Processador	NEC VR5432	NEC VR5000	IDT 32334	IDT 79RC32364
Kernel de autocorrelação	1,0	2,1	1,1	2,7
Kernel de codificador convolucional	1,0	1,9	1,2	2,4
Kernel de alocação de bit de ponto fixo	1,0	2,0	1,2	2,3
Kernel de FFT complexo de ponto fixo	1,0	1,1	2,7	1,8
Kernel de decodificador Viterbi GSM	1,0	1,7	0,8	1,1
Média geométrica dos cinco kernels	1,0	1,7	1,4	2,0

FIGURA A.30 Tamanho de código relativo para o compilador C Apogee Software Versão 4.1 para aplicação da Telecom dos benchmarks EEMBC. As arquiteturas de conjunto de instruções são praticamente idênticas, embora os tamanhos de código variem por fatores de 2. Esses resultados foram relatados de fevereiro a junho de 2000.

Em que pesem essas grandes dificuldades, a arquitetura 80x86 foi enormemente bem-sucedida. As razões são três: 1) sua seleção como o microprocessador no IBM PC inicial torna a compatibilidade binária 80x86 extremamente valiosa; 2) a lei de Moore forneceu recursos suficientes para os microprocessadores 80x86 traduzirem para um conjunto de instruções RISC interno e, depois, executarem instruções tipo RISC. Esse mix permite a compatibilidade binária com a valiosa base de software PC e desempenho compatível com processadores RISC; 3) os altíssimos volumes dos microprocessadores de PC significam que a Intel pode facilmente pagar pelo maior custo de projeto da tradução de hardware. Além disso, os altos volumes permitem que o fabricante suba a curva de aprendizagem, o que baixa o custo do produto.

O maior tamanho do die e o aumento do consumo de potência para translação são problemas para as aplicações embarcadas, mas isso faz um tremendo sentido econômico para o desktop. E sua relação custo-desempenho no desktop também o torna atraente para servidores, com sua principal fraqueza para servidores de 32 bits de endereços, o que foi resolvido com os endereços de 64 bits do AMD64 (Cap. 2).

Falácia. *Você pode projetar uma arquitetura sem falhas.*

Todo projeto de arquitetura envolve compromissos feitos no contexto de um conjunto de tecnologias de hardware e software. Com o passar do tempo, essas tecnologias costumam mudar e as decisões que podem ter sido corretas na época em que foram tomadas parecem ter sido equivocadas. Por exemplo, em 1975, os projetistas do VAX enfatizaram demais a importância da eficiência de tamanho de código, subestimando a importância que a facilidade de decodificação e o pipelining teriam cinco anos depois. Um exemplo no campo do RISC é o desvio atrasado (delayed branch) (Apêndice K). Foi uma simples questão de controlar os hazards de pipeline, com pipeline de cinco estágios, mas foi um problema para processadores com pipelines mais longos que terminam a execução de múltiplas instruções por ciclo de clock. Além do mais, quase todas as arquiteturas posteriormente sucumbem à falta de espaço de endereço suficiente.

Em geral, evitar essas falhas em longo prazo provavelmente significaria comprometer a eficiência da arquitetura em curto prazo, o que é arriscado, já que uma nova arquitetura de conjunto de instruções precisa lutar para sobreviver aos seus primeiros anos.

A.11 COMENTÁRIOS FINAIS

As primeiras arquiteturas eram limitadas em seus conjuntos de instruções pela arquitetura de hardware da época. Tão logo a tecnologia de hardware permitia, os arquitetos de computadores começavam a procurar meios de oferecer suporte a linguagens de alto nível. Essa procura levou a três períodos distintos de pensamento sobre como dar um suporte eficiente aos programas. Na década de 1960, as arquiteturas de pilha se tornaram populares. Elas eram vistas como uma boa opção para linguagens de alto nível — e provavelmente eram, dada a tecnologia dos compiladores da época. Na década de 1970, a principal preocupação dos arquitetos era como reduzir custos de software. Essa preocupação foi satisfeita principalmente substituindo software por hardware ou fornecendo arquiteturas de alto nível que poderiam simplificar a tarefa dos projetistas de software. O resultado foi migração para arquitetura de computador de linguagem de alto nível e para arquiteturas poderosas como VAX, que tinha grande número de modos de endereçamento, diversos tipos de dados e uma arquitetura altamente ortogonal. Na década de 1980, uma tecnologia de compilador mais sofisticada e uma nova ênfase no desempenho do processador provocaram um retorno às arquiteturas mais simples, com base principalmente no estilo carregamento-armazenamento de computador.

As seguintes mudanças de arquitetura de conjunto de instruções ocorreram na década de 1990:

- *Duplicações no tamanho de endereço.* Os conjuntos de instruções de endereço de 32 bits para a maioria dos processadores de desktops e servidores foram estendidos para endereços de 64 bits, aumentando o tamanho dos registradores (entre outras coisas) para 64 bits. O Apêndice K fornece três exemplos de arquiteturas que foram de 32 bits para 64 bits.
- *Otimização dos desvios condicionais através de execução condicional.* Nos Capítulos 2 e 3, vimos que os desvios condicionais podem limitar o desempenho dos projetos agressivos de computador. Portanto, houve interesse em substituir os desvios condicionais por conclusão condicional das operações, como movimentação condicional (Apêndice H), o que foi acrescentado na maioria dos conjuntos de instruções.
- *Otimização do desempenho da cache através de pré-busca (prefetch).* O Capítulo 2 explica o papel cada vez maior da hierarquia de memória no desempenho dos computadores, com falta de cache em alguns computadores exigindo tantos tempos de instrução quantas falhas de página nos computadores antigos. Portanto, as instruções de pré-busca foram acrescentadas para tentar ocultar o custo das faltas de cache pela realização da busca prévia (Cap. 2).
- *Suporte para multimídia.* A maioria dos conjuntos de instruções de desktops e embarcados foi estendida com suporte para aplicações de multimídia.
- *Operações de ponto flutuante mais rápidas.* O Apêndice J descreve as operações acrescentadas para melhorar o desempenho de ponto flutuante, como operações que realizam uma multiplicação e uma adição com uma única execução (essas operações são incluídas no MIPS).

Entre 1970 e 1985, muitos achavam que o principal trabalho do arquiteto de computador era o projeto de conjuntos de instruções. Como resultado, os livros dessa época enfatizam o projeto de conjunto de instruções, assim como os livros das décadas de 1950 e 1960 enfatizam a aritmética de computador. Esperava-se que o arquiteto inteligente tivesse opiniões sobre os pontos fortes e, especialmente, os pontos fracos dos computadores comuns. A importância da compatibilidade binária em anular inovações no projeto de conjunto de instruções não era reconhecida por muitos pesquisadores e escritores de livros, dando a impressão de que muitos arquitetos teriam a chance de projetar um conjunto de instruções.

A definição de arquitetura de computador foi expandida para incluir o projeto e a avaliação do sistema de computador inteiro — não apenas a definição do conjunto de instruções e não apenas o processador — e, portanto, existem muitos tópicos para o arquiteto estudar. Na verdade, o assunto deste apêndice foi um ponto central do livro em sua primeira edição em 1990, mas agora está incluído em um apêndice principalmente como material de referência!

O Apêndice K pode satisfazer os leitores interessados na arquitetura de conjunto de instruções: ele descreve uma variedade de conjuntos de instruções, que são importantes no mercado de hoje ou historicamente importantes, e compara nove computadores de carregamento-armazenamento conhecidos com o MIPS.

A.12 PERSPECTIVA HISTÓRICA E REFERÊNCIAS

A Seção L.4 (disponível on-line) apresenta um estudo sobre a evolução dos conjuntos de instruções e inclui referências de leitura complementar e exploração de tópicos relacionados.